

UTA MACS — HOTO & Vendor Management Platform Design

v4.1 — 4-Role Model + committee_title Separation

Society: Urban Trilla Apartment Owners Mutually Aided Cooperative Maintenance Society Limited

Registration No: TG/RRD/MACS/2026-15/FOW & M (registered 10-02-2026)

Location: SY NO:425/2/1, Kondakal Village, Shankarpally Mandal, Rangareddy District, Telangana

Builder (Promoter): Ankura Homes | **HOTO Consultant:** Ascenza Global Infra Care Pvt Ltd

HOTO Start Date: June 1, 2026 | **Maintenance Tracking From:** May 1, 2025

Document Version: 4.1 — May 2026 (role simplification: 9 roles → 4 roles + committee_title display field)

Table of Contents

1. [What We Are Building and Why](#)
2. [Byelaw Governance Rules Hardcoded into the System](#)
3. [System Architecture](#)
4. [Infrastructure Resilience Design](#)
5. [Module 0 — User & Role Management](#)
6. [Module 1 — HOTO Management](#)
7. [Module 2 — Snag List Management](#)
8. [Module 3 — Vendor Evaluation & Selection](#)
9. [Module 4 — Financial Tracking](#)
10. [Module 5 — Formal Notice Generation](#)
11. [Workflow Engine & Approval Delegation](#)
12. [Non-Tech User Experience Specification — §12.5 Validation Messages & Blocked Feature UX](#)
13. [Dashboard & UX Design](#)
14. [Git Storage Strategy](#)
15. [Data Model](#)
16. [Security & Privacy Compliance](#)
17. [Data Migration Sprint](#)
18. [Role-Based Access Control & Feature Permissions](#)
19. [Document Management](#)
20. [Scope Boundary](#)
21. [Phase-wise Implementation Plan — §21.0 Non-Regression Principles](#) applies to every sprint task
22. [Comprehensive Risk Register](#)
23. [Rules Engine](#)
24. [Email Management & Draft System](#)
25. [RBAC Administration UI — Complete Specification](#)
26. [Post-Redesign Regression Analysis](#)

Operations: See [design/RUNBOOK.md](#) for step-by-step operations procedures.

1. What We Are Building and Why

Urban Trilla MACS has 136 units (40-50 currently occupied), 14 committee members, and is entering the most consequential phase of a cooperative society — the Handover/Takeover from builder Ankura Homes. The HOTO process starts June 1, 2026, has a 45-day Ascenza-led audit timeline, and is expected to span 2-3 months depending on builder responsiveness.

The problem today: All evidence, decisions, communications, and tracking live in WhatsApp messages, personal emails, Google Drive folders, and physical files. The two most senior decision-makers (President Bal Reddy and Working President) are non-technical users who are comfortable with WhatsApp. For the system to succeed, it must be simpler than a WhatsApp group in terms of mental load.

The three core pillars:

1. **Radical simplicity** — President and Working President can use it without training
2. **Complete auditability** — every action permanently recorded; nothing disappears
3. **Byelaw compliance** — governance rules hardcoded, not configurable

What v3.1 adds over v3:

Addition	Why it matters
Admin as dedicated system role	Admin is the technical executor of user/permission management — acts on documented consent from President or Secretary; not a governance role
Authorization-documented admin actions	Every admin action on roles or permissions must record who in leadership authorized it, how (WhatsApp/email/verbal/meeting), and when — creates defensible audit trail without burdening non-tech leaders with portal logins
Module 0: User & Role Management	Without controlled user access, every other module's security is theoretical
Invite-only registration	Prevents unauthorized access to governance documents
Committee election bulk update	Annual elections change 14 roles simultaneously; manual one-by-one is error-prone
Role change with audit trail	Every role change must be logged for byelaw audit requirements
Feature permission system	Admin enables/disables individual features per role (with President authorization documented)
Per-user feature overrides	Edge cases where one person needs temporary access to a module
UI feature gating	Every button/section renders conditionally based on the logged-in user's actual permissions
Fix existing <code>user_roles</code> bug	Current table missing UNIQUE constraint — role changes silently fail

What v3 added over v2 (unchanged):

Risk from analysis	Design response
Vercel 10s timeout kills PDF	Async PDF generation with job queue; Vercel Pro before HOTO
GitHub API limits on bulk upload	<code>upload_queue</code> table; cron-processed batches
GitHub token silent failure	<code>github_api_log</code> + health-check + Resend alert
Supabase free tier pauses DB	Cron ping every 6 days
President doesn't adopt	Zero-ambiguity mobile screen; pre-launch walkthrough
WhatsApp shortcuts governance	Financial payments require portal approval record

Risk from analysis	Design response
Snag scope confusion	<code>snag_scope</code> field; liability disclaimer
RERA document metadata	Server timestamps, SHA-256 hash, source description
Corpus fund overdraft	Server-side balance check
Builder SLA drift	Escalation cron at 7/14/30 days overdue
Phase 1 too late for June 1	Emergency sprint plan: minimum viable system live May 31

2. Byelaw Governance Rules Hardcoded into the System

These are legal requirements under registered Byelaws (Reg No: TG/RRD/MACS/2026-15/FOW & M). They are not configurable by any admin.

2.1 Voting Rules

Rule	Byelaw Reference	System Implementation
One apartment = one vote	§4.16	Each member gets exactly 1 vote; no weighting by role
Cannot vote if >90 days maintenance arrears	§4.6	System checks <code>payment_status</code> before showing vote button
Board decisions by majority vote	§7.16(c)	Simple majority of votes cast
President has casting vote on tie	§7.16(c) & §8.1	If tied, President casting vote logged with byelaw citation
Board quorum = simple majority of directors	§7.16(a)	With 14 directors, minimum 8 must vote
Member can authorize via registered PoA	§4.16	PoA document uploaded + linked to vote record
Joint ownership voting	Policy per §4.16	First named owner votes; policy committed to governance-data pre-vote

2.2 Decision Approval Chain

Scenario	Byelaw Reference	System Rule
HOTO/vendor approvals require dual sign-off	§8.1 + §8.3	Both President AND Secretary must approve
President absent (planned, >7 working days)	§8.2	Admin sets delegation → VP; all VP actions tagged "per §8.2"
President absent (unplanned/urgent)	§8.2	VP may act; flagged for President review on return
Secretary absent (planned)	§8.4	Joint Secretary takes all Secretary functions
Both President and VP unavailable	None	System freezes approval gates

2.3 Financial Authority Limits

Authority	Limit	Byelaw Reference	System Rule
Secretary (urgent remedial)	Up to ₹10,000/-	§9.11(a)	API enforces; rejects above
President (urgent remedial)	Up to ₹20,000/-	§9.11(a)	API enforces; rejects above
Board of Directors	Up to ₹50,000/-	§9.11(b)	Requires Board vote with quorum
Beyond ₹50,000/-	General Body required	§9.11(b)	API blocks with message
All payments >₹10,000/-	Must be electronic	§9.11(c)	Warning if non-electronic indicated
Cash payments	Prohibited	§5.3(p) & §9.1	Cash option removed entirely
Corpus fund overdraft	Prohibited (policy)	—	Server-side balance check in payment API

2.4 Conflict of Interest

Rule	Byelaw Reference	System Implementation
Director must not participate where personally interested	§7.16(b)	Recuse button before every vote; recusal permanent and logged
Office bearers receive no remuneration	§3.4(b)	Vendor with committee member interest is flagged

2.5 Transparency & Records

Rule	Byelaw Reference	System Implementation
Minutes within 7 days of Board meeting	§7.16(e)	Upload tracker; dashboard flag
Defaulter list published monthly	§9.6	Auto-generated; first Sunday each month
Financial statements by 30th September	§9.3	Dashboard reminder from Sep 1
Data retention: 10 years	Requirement	Git history = permanent

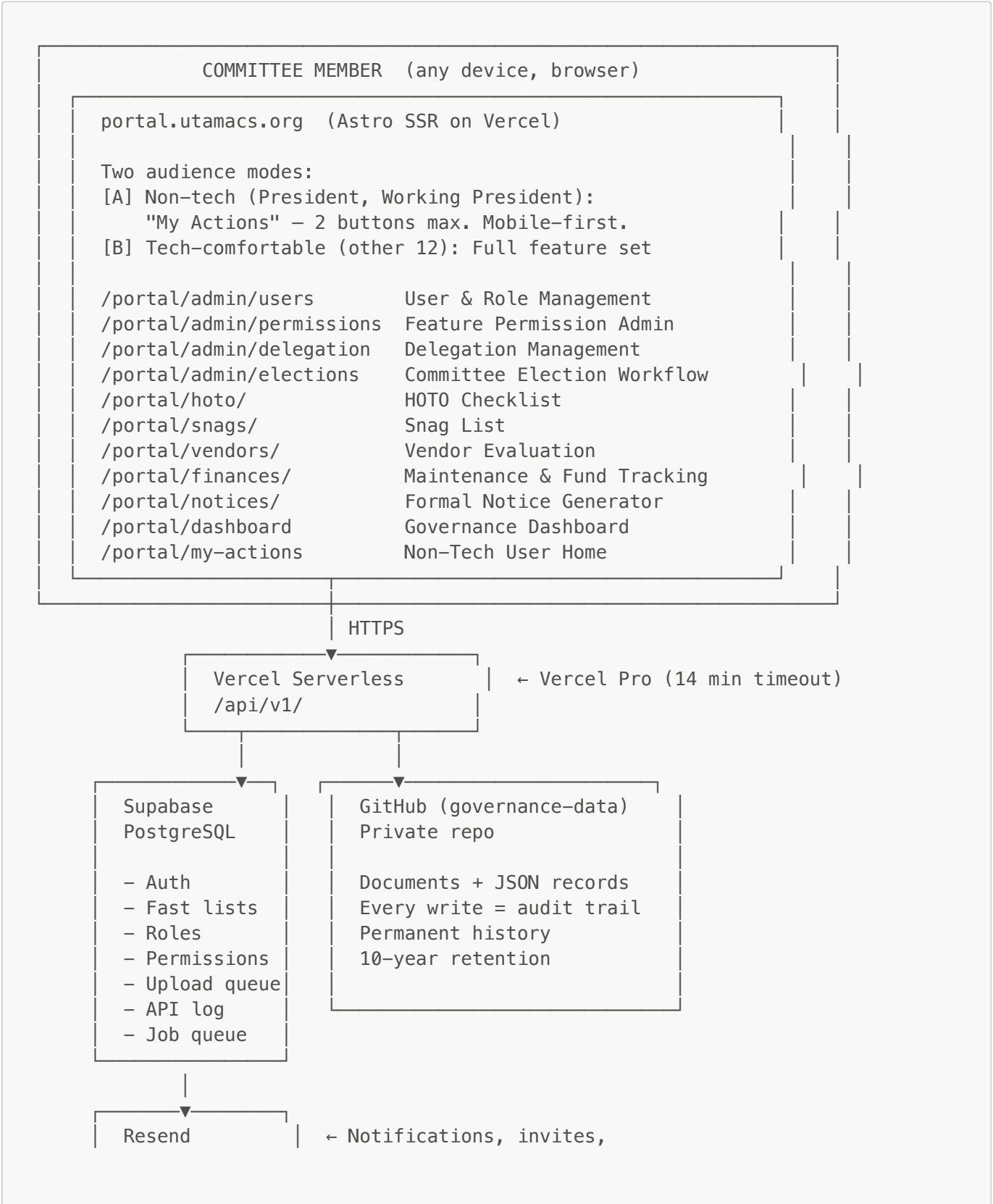
2.6 Defaulter Rules

Rule	Byelaw Reference	System Flag
2 months arrears = Defaulting Member	§6.36	Yellow flag at 60 days
3 months arrears = services can be denied	§6.37	Red flag at 90 days; 7-day notice countdown
18% per annum interest	§19(e)	Auto-calculated from due date

Rule	Byelaw Reference	System Flag
Vote rights suspended at 90 days	§4.6	payment_status = defaulter_90d blocks vote button

3. System Architecture

3.1 Architecture Overview



(email)

health alerts, digests

3.2 Two Repositories

utamacs/utamacs-website ← Code (public – portal.utamacs.org)

utamacs/governance-data ← Private data repo (documents + audit trail)

3.3 Key Design Constraints

1. **No direct browser-to-GitHub upload.** All uploads: Browser → API → **upload_queue** → cron → GitHub.
2. **No PDF generation in the request cycle.** PDF jobs: Browser → API → **pdf_generation_jobs** → cron → stored PDF → download URL.
3. **All files in GitHub, never Supabase storage.** Supabase storage = 500MB limit; GitHub = no limit.
4. **Every consequential action has a server-side guard.** Financial limits, voting eligibility, quorum, balance check — all enforced in API routes, never in client-side JavaScript.
5. **Feature permissions are enforced at two levels.** UI: buttons/sections conditionally rendered per role. API: every route independently re-checks permissions. UI hiding is UX; API checking is security.

3.4 Committee Structure Mapped to Roles

The role is authority. The title is a label. Only four **portal_role** values exist in the system. They correspond to functionally distinct approval and financial authority levels required by the byelaws. All other committee designations are stored as a human-readable **committee_title** field — visible in the directory and on profile pages, but with zero effect on permissions or system logic.

The Four Governance Roles

portal_role	Financial authority	Approval authority	Who holds it
president	Up to ₹20,000 (§9.11a)	Final HOTO gate; casting vote (§7.16c)	President
secretary	Up to ₹10,000 (§9.11a)	HOTO co-approval gate; notices; finance open	General Secretary
executive	None	Create/edit/upload/comment/vote	All other 12 committee members
member	None	Read-only	General apartment owners

Committee Titles (display only — no effect on permissions)

Committee person	portal_role	committee_title	Special handling
President	president	"President"	—
Vice President	executive	"Vice President"	Designated as delegation target for president-absence (§8.2)

Committee person	portal_role	committee_title	Special handling
Working President	executive	"Working President"	—
General Secretary	secretary	"General Secretary"	—
Joint Secretary	executive	"Joint Secretary"	Designated as delegation target for secretary-absence (§8.4)
Treasurer	executive	"Treasurer"	Finance features (view + enter) enabled via user feature override
Joint Treasurer	executive	"Joint Treasurer"	Finance features enabled via user feature override (if needed)
Executive Member (x7)	executive	"Executive Member"	—
General Member	member	null	—

Why this collapses 9 roles to 4

Of the previous 9 roles:

- **working_president** — identical capabilities to executive; title only
- **vice_president** — executive in base state; gains president powers only when delegation is activated (§8.2) — handled by the **approval_delegations** table, not a role
- **joint_secretary** — executive in base state; gains secretary powers only when delegation is activated (§8.4) — same mechanism
- **treasurer** — executive + **finance.view/finance.enter** features; these are feature permissions, not a distinct approval authority
- **joint_treasurer** — effectively identical to executive

Only **secretary** and **president** have unique, byelaw-mandated approval authority that cannot be replicated by feature permissions. They remain as distinct roles.

Admin flag (orthogonal to governance role)

Flag	Who holds it	What it enables
is_admin = true	Designated system admin	Full user management, feature permissions, election workflow, delegation management

A person can be: admin-only (non-committee tech admin), executive + admin, secretary + admin, or president + admin. No governance power comes from **is_admin** — only system management capability.

4. Infrastructure Resilience Design

4.1 GitHub Upload Queue

All uploads are queued and processed asynchronously by a cron job (max 30 files per run, every 60 seconds).

```

Browser → POST /api/documents/upload
→ File in Vercel /tmp
→ Record in upload_queue (status: PENDING)
→ Returns { queue_id }
→ Browser polls GET /api/documents/status/[queue_id]

Cron /api/cron/process-uploads (every 60s):
→ Takes ≤30 PENDING items
→ Commits to GitHub via GitHub App
→ COMPLETED: inserts to documents table
→ FAILED (3 retries): PERMANENTLY_FAILED; alert email sent

```

Upload constraints:

- Max 5MB per file (enforced server-side)
- Accepted types: PDF, JPG, PNG, XLSX, CSV, DOCX
- No video — `video_url` field for YouTube/Drive links only

4.2 GitHub API Health Monitor

```

Every 15 minutes (cron /api/cron/github-health):
→ GET governance-data/README.md via GitHub API (read-only – zero commits
created)
→ Log result + latency to github_api_log
→ On 3 consecutive failures: Resend alert to Secretary + admin

Every 6 days (cron /api/cron/supabase-ping):
→ SELECT 1 FROM profiles LIMIT 1
→ Prevents Supabase free-tier 7-day pause

```

Why read-only? A write-based health check (creating a commit on every 15-minute ping) would generate 96 commits per day — polluting the governance audit trail and counting against GitHub API rate limits. A `GET /contents/README.md` call verifies connectivity and token validity without any side effects.

Circuit breaker integration: On 3 consecutive failures, the circuit breaker opens (see §4.5) — all upload processing stops until the check starts passing again. Recovery is automatic; no admin intervention required.

4.3 Async PDF Generation

```

User clicks "Generate PDF"
→ POST /api/pdf/generate → inserts pdf_generation_jobs (status: QUEUED)
→ Returns { job_id } → UI shows spinner
→ Cron every 30s processes QUEUED jobs
→ PDF stored in GitHub → job status = DONE
→ UI polls, gets DONE, shows [Download] button

```

Vercel Pro upgrade (14-minute timeout) is the backup safety net.

4.5 Async Resilience Patterns

Every async operation can fail. The system handles failures in layers: retry → circuit break → dead letter queue → human intervention. No failure silently disappears.

Retry Policy (Exponential Backoff)

Attempt	When	On failure
1	Immediate (first try)	Set <code>backoff_until = NOW() + 5 min</code>
2	5 minutes later	Set <code>backoff_until = NOW() + 30 min</code>
3	30 minutes later	<code>status = PERMANENTLY_FAILED</code> → DLQ alert to admin

Fixed-interval retries hammer a recovering service. Exponential backoff gives it time to stabilise. Upload queue and PDF generation both use this pattern.

```
-- Cron query selects only items past their backoff window:
SELECT * FROM upload_queue
WHERE status = 'PENDING'
      AND (backoff_until IS NULL OR backoff_until < NOW())
ORDER BY created_at LIMIT 30;
```

Circuit Breaker

If `github_api_log` has 3 consecutive `success = false` entries, the circuit breaker opens. All upload processing stops to avoid hammering a recovering service.

```
State: CLOSED (normal)
  → 3 consecutive health-check failures
State: OPEN (blocked)
  → All upload_queue processing skipped
  → Admin banner: "Document storage unavailable. Uploads paused."
  → Every 5 min: read-only GET to check recovery
  → On read success:
State: CLOSED (recovered)
  → Trigger immediate on-demand cron run with 3× batch size (90 items)
  → Admin banner: "Storage restored. Processing queued uploads."
```

Circuit breaker state is stored in `system_config` table (key: `github_circuit_breaker`, values: `OPEN/CLOSED`).

Idempotency

Vercel Cron does not guarantee exactly-once delivery. Every cron run is protected by a distributed lock:

```
CREATE TABLE cron_locks (
  item_type TEXT NOT NULL,
  item_id TEXT NOT NULL,
  run_id UUID NOT NULL,
  acquired_at TIMESTAMPTZ DEFAULT NOW(),
  expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '10 minutes',
  PRIMARY KEY (item_type, item_id)
);
```

```
-- Before processing any item:
INSERT INTO cron_locks (item_type, item_id, run_id)
VALUES ('upload', $upload_id, $run_id)
ON CONFLICT DO NOTHING;
-- 0 rows inserted → item is being processed by another run → skip
-- Locks auto-expire after 10 min (safeguard for crashed runs)
```

Cron Heartbeat Monitoring

Every cron writes a heartbeat. A daily job compares last-run time to expected interval × 2 and alerts admin if a cron has gone silent (Vercel Cron can fail to fire).

Cron	Expected interval	Alert threshold
process-uploads	60 s	5 min silent
github-health	15 min	45 min silent
process-pdfs	30 s	5 min silent
builder-sla	24 h	36 h silent
supabase-ping	6 days	8 days silent
pdf-purge	24 h	36 h silent

Dead Letter Queue (DLQ) Dashboard — /portal/admin/queue

Admin-only page showing system health and all PERMANENTLY_FAILED items:

QUEUE HEALTH

Last updated: 2 min ago

Github Storage: ✔ Connected

Circuit Breaker: ✔ Closed

Upload Queue: 0 pending · 0 in-progress · 2 ⚠ failed

CRON STATUS

✔ process-uploads

last run: 42s ago

✔ github-health

last run: 7m ago

⚠ builder-sla

last run: 38h ago

[Trigger Manually]

✔ supabase-ping

last run: 4d ago

DEAD LETTER QUEUE (2 items):

⚠ PERMANENTLY FAILED

hoto/HOTO-042/documents/kone-amc.pdf

Uploaded by: Treasurer · 3 days ago

Error: GitHub API 422 – blob too large (> 100MB GitHub limit)

[↩ Retry] [x Abandon] [↓ Download Original]

- **Retry:** Resets `status = 'PENDING', attempts = 0, backoff_until = null`
- **Abandon:** Sets `status = 'ABANDONED'`; sends email to original uploader: "Your file could not be saved. Please re-upload."
- **Download Original:** Retrieves from Vercel `/tmp` if still cached (best-effort within 24h)

10 / 82

4.4 Non-Developer Operations Runbook

RUNBOOK.md is a separate document at [design/RUNBOOK.md](#) and committed to **governance-data** before go-live. Covers: adding members, resetting passwords, activating/deactivating delegation, checking upload queue, rotating GitHub App key, recovering from storage and database failures, managing the email draft queue, updating rules, monitoring cron jobs, and portal-down procedures. Validated by Secretary following it without assistance before go-live.

5. Module 0 — User & Role Management

This module is the foundation for everything else. Every action in the platform is attributed to a user with a specific role, and every button or section in the UI is conditionally rendered based on that user's permissions. Without correct role management, the approval chains, audit trails, and access controls all break down.

5.1 The Admin Role: Executor with Documented Authorization

Core principle: The President and Secretary make governance decisions. The admin executes them in the system. The admin never acts unilaterally on user access or permissions — every significant action requires documented consent from the President or Secretary.

Why this separation?

- The President and Working President are non-technical. They should not need to log into an admin panel to approve every user addition or role change.
- The admin (typically the implementer or a tech-savvy designated person) handles the technical execution.
- But because the admin has broad system access, every action must have a documented authorization trail — who in leadership approved it, how they communicated that (WhatsApp/email/meeting), and when.

Who is the admin?

- A designated person with `profiles.is_admin = true`
- Typically: the system implementer initially; later transferred to a tech-comfortable committee member
- Can hold a governance role simultaneously (e.g., admin + executive) or be admin-only (non-committee)
- The admin flag is set by another admin (or by the implementer at setup)
- A society can have more than one admin; each admin's actions are independently logged

What the admin can do (with documented authorization):

- Invite new members and committee members
- Change governance roles
- Run committee election bulk updates
- Grant or revoke per-user feature permission overrides
- Manage feature permissions per role
- Activate/deactivate delegation chains
- Deactivate or reactivate members
- Manage the admin flag itself (grant/revoke `is_admin` for others)

What the admin cannot do (no matter their technical access):

- Cast votes on vendor decisions (unless they also hold a governance role that allows voting)
- Approve HOTO items (unless they also hold president/secretary governance role)
- Act on behalf of President or Secretary in governance decisions

5.2 Admin Acts Unilaterally — No In-Portal Approval Required

The admin has full authority to manage users and permissions without needing to wait for or collect a digital approval from the President or Secretary inside the portal. The expectation is that the President/Secretary communicate decisions to the admin via their normal channels (WhatsApp, phone, meeting) — the admin then acts.

Why no in-portal authorization workflow?

- The President and Working President are non-technical. Requiring them to log into an admin panel and click "Approve" on every user change adds friction that defeats the purpose.
- The admin is a trusted, designated role. Assigning `is_admin = true` to someone is itself the act of trust — it should be done carefully once, not verified on every subsequent action.
- The audit log records everything the admin does. If a decision is ever questioned, the audit trail shows what changed, when, and by whom.

What keeps the admin accountable:

- Every admin action is permanently logged in `audit_log` (who, what, when)
- The user's role history timeline is visible to the President, Secretary, and Vice President — they can review changes at any time
- The admin cannot change their own role or grant themselves extra governance powers
- Only another admin (or the initial setup) can grant the `is_admin` flag to someone

5.3 Registration Model: Invite-Only

No one can self-register. All portal access starts with an admin-sent invite — initiated only after the admin receives authorization from the President or Secretary.

Registration Flow:

1. Admin: `/portal/admin/users` → [Invite Member]
Enter: email, flat number, intended role (default: member)
2. System:
 - Creates `member_invites` record with one-time token (expires 7 days)
 - Sends Resend email with invite link + flat number + portal introduction
3. New user clicks link:
 - Registration form: name + password (email pre-filled, non-editable)
 - Privacy consent checkbox (DPDP Act — must accept to proceed)
 - On submit: account created, `privacy_consent` record saved
4. Admin notified: "[Name] (Flat 207) has accepted their invitation"
Secretary also notified via email (FYI — not for approval)
5. `member_invites.accepted = true`; token immediately invalidated

Profile creation timing (FK safety): Supabase Auth creates the `auth.users` record first, which triggers a database trigger that immediately creates the corresponding `profiles` row. Only *after* the `profiles` row exists does the registration API set `member_invites.accepted_user_id`. This is enforced by wrapping both operations in a Postgres function called from the API route — the `profiles` INSERT fires via trigger, then the `member_invites` UPDATE runs in the same transaction. There is no window where `accepted_user_id` is set before the FK target exists.

Token security: The token is compared using constant-time string comparison (`crypto.timingSafeEqual` in Node.js) to prevent timing attacks. The raw token value is never written to application logs.

Invite expiry: Admin can resend from Pending Invites tab. **Invite cancellation:** Admin can cancel; cancelled invites cannot be accepted.

5.4 Role Hierarchy (Governance Roles)

```
member → executive → secretary → president
```

Four roles. Each level adds capability; no capability is removed when moving up. The `is_admin` flag is orthogonal — not in this hierarchy.

What changes when you move up:

From	To	What is gained
member	executive	Create/edit/upload/comment/vote on all governance items
executive	secretary	HOTO co-approval gate; ≤₹10K financial authority; open Board votes; send notices; view member phones
secretary	president	HOTO final approval gate; ≤₹20K financial authority; casting vote; delete snags; manage delegation

committee_title is separate from **portal_role** — it is a display-only label set by the admin when assigning or changing a person's role. The title "Vice President" does not grant any additional permissions; it is a human-readable indicator of the person's committee designation.

Rule: The admin assigns both **portal_role** and **committee_title**. No user — including the admin — can change their own role.

5.5 Role Change Workflow

- Admin: `/portal/admin/users` → Click user → [Change Role]
- Two fields to set:
 - Portal role: [member ▼ / executive ▼ / secretary ▼ / president ▼]
 - Committee title: [text field — e.g. "Vice President", "Treasurer", "Executive Member"]
Suggested titles shown as autocomplete options; free text accepted.
- Enter reason (free text — for the audit trail):
 "Committee election — June 2026 AGM"
 "Secretary resigned — promoted from executive"
 "New apartment owner onboarded"
- If role is set to executive and title is "Treasurer" or "Joint Treasurer":
 System prompts: "This person is being designated as Treasurer. Do you want to enable finance access (view records + enter records) for this user? [Yes, enable finance access]
 [No, I'll do this separately]"

5. Confirmation dialog shows both changes:
"You are changing [Name]:
Role: executive → secretary
Title: 'Executive Member' → 'General Secretary'
This takes effect immediately and is permanently logged. Continue?"
6. On confirm (API /api/admin/users/[id]/role PATCH):
→ profiles.portal_role updated
→ profiles.committee_title updated
→ role_change_log record created (records both portal_role AND committee_title change)
→ audit_log record created
→ All assigned items reviewed for auto-reassignment (by portal_role)
→ User receives email: "Your UTA MACS access has been updated.
Role: Secretary | Title: General Secretary"
→ Secretary notified (FYI) – unless they are the person being changed

Title-only changes (no role change): admin can update `committee_title` without changing `portal_role`. This creates a `role_change_log` entry with `old_role = new_role` and a `title_change` note. Audit trail is preserved.

Role changes are **immediate**. The reason field provides enough context for the audit trail without requiring a separate approval workflow.

5.6 Committee Election Bulk Update

The admin runs this workflow after the General Body election concludes. The President or Secretary communicates the election outcome — the admin then executes role and title changes.

In the 4-role model, most election changes are **title changes only** (the person remains `executive`). Role changes are needed only when someone gains or loses Secretary or President authority.

Election Workflow at `/portal/admin/elections`:

- Step 1: [New Election]
Enter: Election date, Description ("Annual General Body Meeting 2026")
Attach: AGM minutes document (optional but recommended)

Step 2: System shows current committee lineup with TWO columns: Role + Title

COMMITTEE ELECTION – 15 June 2026 Update roles and titles. Unassigned members revert to 'member'.		
Committee Position	Current Holder	New Holder
President (role: president)	Bal Reddy	[Bal Reddy ▼]
Vice President (role: executive)	[Name] "Vice President"	[Select member ▼]
Working President (role: executive)	[Name] "Working President"	[Select member ▼]
General Secretary (role: secretary)	[Name]	[Select member ▼]

Joint Secretary (role: executive)	[Name] "Joint Secretary"	[Select member ▼]
Treasurer (role: executive)	[Name] "Treasurer"	[Select member ▼]
Joint Treasurer (role: executive)	[Name] "Joint Treasurer"	[Select member ▼]
Executive Member x7 (role: executive)	[Names]	[Select members ▼] x7

Step 3: Preview screen – clearly separates role changes from title-only changes

```

ROLE CHANGES (permission level changes – 2 people):
[Name A] executive → secretary | title: "Exec Member" → "Gen Sec" |
[Name B] secretary → member    | (outgoing – not re-elected)

TITLE-ONLY CHANGES (no permission change – 8 people):
[Name C] executive, "Joint Secretary" → "Treasurer"
[Name D] executive, "Treasurer" → "Joint Treasurer"
[Name E] executive, – → "Vice President"
... (5 more)

FINANCE ACCESS (prompted automatically):
[Name C] is now titled Treasurer – enable finance.view, finance.enter?
[Yes ✓] [No, handle separately]

[Cancel] [Confirm Election]
```

Step 4: On confirm (single database transaction):

- All portal_role and committee_title changes atomically
- All changes linked to election_event_id
- Finance feature overrides created if confirmed above
- Old designation holders who are not re-elected: title cleared, delegation targets updated
- Each affected person receives email:
 - "Role: Executive | Title: Vice President" or
 - "Role: Secretary | Title: General Secretary"
- Secretary and President notified (FYI)

Why atomic? A partial failure leaves the system in an inconsistent state (e.g., new Secretary appointed but old one still holds the role). Either the full election applies or nothing changes.

After the election — delegation targets: If the VP person changed, the admin must also update the delegation configuration (Admin → Delegation) to point to the new VP. The system prompts this: "The Vice President designation changed. Update the President-absence delegation target? [Go to Delegation Settings]"

5.7 Member Deactivation

When an apartment owner sells their flat (NOC process complete), the admin deactivates them:

1. Admin: /portal/admin/users → Find member → [Deactivate]

2. Enter reason: "Flat 204 sold – NOC issued 2026-06-15"

3. On confirm:

→ profiles.is_active = false

→ All active sessions immediately invalidated

→ Email sent to deactivated member: "Your UTA MACS portal access has been deactivated"

→ All their assigned HOTO/snag items auto-reassigned

→ Secretary and President notified (FYI)

→ Data retained for 10-year audit requirement

Reactivation: Admin only; mandatory reason.

Account deletion: Never. Data retention is a byelaw requirement (10 years).

5.8 Auto-Reassignment on Role Loss

When a user loses a committee role (downgraded or deactivated), all items assigned to them are automatically reassigned. Priority order:

1. Item has responsible_role set?

→ Find current holder of that role → reassign to them

2. The role itself is currently vacant?

→ Escalate to Secretary

3. Secretary also unavailable?

→ Escalate to President

4. President also unavailable?

→ System flags item as "Needs manual assignment" on dashboard

All auto-reassignments logged in audit_log:

action: "AUTO_REASSIGNED"

reason: "Role change: [old user] downgraded from [role]"

5.9 User Directory (/portal/admin/users)

Visible to: **admin** (full access) + secretary, president (read-only view)

The directory shows both portal_role (permission badge) and committee_title (display label) as separate pieces of information. The role badge is the system authority; the title is the human-readable designation.

MEMBER DIRECTORY					[+ Invite Member]	
Filter: [All Roles ▼] [All Status ▼] Search: [_____]						
Tabs: [Active (14)] [Pending Invites (2)] [Inactive (1)]						
Name	Flat	Role & Title		Payment	Actions	

Bal Reddy	101	 President "President"	 Current	[View]
[Name]	204	 Secretary "General Secretary"	 Current	[View]
[Name]	312	 Executive "Vice President"	 45d	[View]
[Name]	108	 Executive "Treasurer"	 Current	[View]
[Name]	215	 Member	 Current	[View]

Role badges (permission level):  President ·  Secretary ·  Executive ·  Member
Title is shown in smaller text below the role badge — no badge, no colour.

Filter by role filters on `portal_role` (the authority level). Filter by title is available as a separate search field for quick lookup (e.g., "find everyone titled Treasurer").

User detail page shows:

- Profile info + flat number + committee_title
- Role history timeline: `member` → `executive` [title: Exec Member] → `executive` [title: Treasurer] → `secretary` [title: General Secretary]
- Note: title changes within the same role are shown as timeline events with a distinct "title change" tag, not a "role change" tag
- All HOTO/snag items assigned to them
- All votes cast; all documents uploaded; all comments
- Their feature permissions (role defaults + user overrides)
- Finance access override (if Treasurer: shows `finance.view` ✓, `finance.enter` ✓ — from override)

What admin sees vs. what leaders see:

- Admin: full directory + all actions (Invite, Change Role & Title, Deactivate, Grant Override)
- President/Secretary (non-admin): read-only directory — can see members, roles, titles, last active; no action buttons

5.8 Pending Invites Tab

PENDING INVITES (3)				[+ Invite Member]
Email	Flat	Role	Sent	Expires / Action
a@b.com	207	member	3 days ago	4 days [Resend]
c@d.com	415	executive	6 days ago	1 day [Resend]
e@f.com	102	member	8 days ago	EXPIRED [Resend]

6. Module 1 — HOTO Management

6.1 HOTO Scope (Ascenza-Aligned Categories)

Category	Ascenza Scope Section
Statutory Compliance	Land docs, OC, NOCs, Fire NOC
Technical - Electrical	LT/HT, DG sets, earthing, common lighting
Technical - Lifts	4 elevators, commissioning, AMC transfer
Technical - Fire & Safety	Hydrant, sprinkler, extinguishers, fire doors
Technical - HVAC	Ventilation, pressurization, exhaust
Technical - Water & Plumbing	DWS/SWS/FWS, STP, WTP, boreholes
Technical - Security & IT	CCTV, access control, boom barriers, intercom
MEP - Miscellaneous	Solar, gas bank, BMS, PA system
AMC Due Diligence	All AMC contracts — status, transfer to association
Snagging	Civil, seepage, exterior, common areas, terrace, club house
Asset/Inventory	Asset register, parking, spare keys
Financial Handover	Corpus fund transfer, maintenance corpus, builder dues

6.2 HOTO Item State Machine

```

NOT_STARTED → IN_PROGRESS → EVIDENCE_UPLOADED → UNDER_REVIEW
               → PENDING_PRESIDENT → PENDING_SECRETARY → APPROVED → COMPLETED
                                   |
                                   └─────────────────── DISPUTED
  
```

State transition role rules (using 4-role model):

- **NOT_STARTED → IN_PROGRESS**: **executive** or above
- **IN_PROGRESS → EVIDENCE_UPLOADED**: **executive** or above (requires ≥1 document)
- **EVIDENCE_UPLOADED → UNDER_REVIEW**: **executive** or above
- **UNDER_REVIEW → PENDING_PRESIDENT**: **secretary** role only
- **PENDING_PRESIDENT → PENDING_SECRETARY**: **president** role only (or **executive** with active president-delegation activated by admin per §8.2)
- **PENDING_SECRETARY → APPROVED**: **secretary** role only (or **executive** with active secretary-delegation activated by admin per §8.4); cannot be same person who set PENDING_PRESIDENT
- **APPROVED → COMPLETED**: **president** or **secretary** only
- **COMPLETED → DISPUTED**: **president** or **secretary** — requires written reason in **governance_notes**

How delegation works in transitions: When the admin activates president-delegation to a specific executive (the VP-designated person), that executive temporarily gains the **hoto.approve_president** feature. The system checks the **approval_delegations** table on every API call — it does not change the person's **portal_role**. The delegation state is clear in the audit trail: every delegated approval is tagged "Acting per §8.2 delegation".

6.3 Builder SLA Escalation

Every builder-dependent HOTO item has a **builder_sla_date**. Cron job checks daily:

7 days overdue → Email committee: "[HOTO-042] Lift AMC is 7 days overdue"
14 days overdue → Email with URGENT flag; dashboard red countdown
30 days overdue → Auto-generate draft formal notice → notify Secretary to review and send

If rera_escalation_eligible = true: status → RERA_ELIGIBLE

6.4 Role-Based Assignment (Turnover-Safe)

Every HOTO item stores both **responsible_role** (e.g., **executive**) and **responsible_user_id**. If the person changes roles, the item stays with whoever currently holds the role. All auto-reassignments logged.

7. Module 2 — Snag List Management

7.1 Snag Scope Classification

Every snag must declare its scope:

snag_scope	Meaning	Liability
COMMON_AREA	Corridor, terrace, basement, lobby, clubhouse, external	Society responsibility post-HOTO
INDIVIDUAL_APARTMENT	Inside a specific flat	Owner-builder matter

For **INDIVIDUAL_APARTMENT** snags, a non-removable banner shows:

"UTA MACS is logging this for reference only. The Society is not a party to this snag's resolution."

Only **COMMON_AREA** snags appear in formal HOTO documentation and builder notices.

7.2 Snag Item States

OPEN → IN_PROGRESS → BUILDER_NOTIFIED → BUILDER_COMMITTED → RESOLVED → VERIFIED_CLOSED

└─ REOPENED

7.3 Features

- **Create:** Any executive or above; must set **snag_scope**, category, location, severity
- **Update:** Any executive or above
- **Delete:** Soft-delete; president only; mandatory **deletion_reason**; permanently logged
- **VERIFIED_CLOSED:** President or Secretary only
- **Reopen:** If a verified-closed snag is reopened, a **reopen_reason** is mandatory — stored in **snag_items.reopen_reason** and audit log
- **Bulk import:** CSV/XLSX from Ascenza punch-list format; column mapping screen; photo ZIP supported

Role-based assignment (snags): Every snag stores both **responsible_role** (e.g., **secretary**) and **responsible_user_id**. If the responsible person's role changes or they are deactivated, the snag auto-reassigns to the current holder of that role using the same auto-reassignment logic as HOTO items (§5.8). All reassignments are logged in **audit_log**.

8. Module 3 — Vendor Evaluation & Selection

8.1 Active Evaluations

Category	Known Vendors
Property Management Platform	MyGate, NoBroker
Accounting/Finance Tool	Mandix, Hari
Facility Management	Kapston, Kapil
Legal Counsel	TBD
Security Vendor	TBD

8.2 Voting Model (§4.16 — One Apartment One Vote)

Each director gets exactly 1 vote. Quorum = 8 of 14 (§7.16(a)). Tie → President casting vote (§7.16(c)). All votes visible.

8.3 Proxy and Joint Ownership

- Proxy: notarized PoA uploaded by admin; linked to vote record
- Joint ownership: first named owner votes by default; policy committed to governance-data pre-vote
- `voting_policy_committed` flag on `vendor_requirements` must be `true` before voting opens

Proxy voting is DISABLED by default. The `PROXY_VOTING_ENABLED` rule (see §23) defaults to `false`. When disabled: the proxy upload option is hidden in the voting UI, and the API rejects any proxy vote submission with: "Proxy voting is not currently enabled for this society. Contact the Admin to enable it if required." Admin can enable it from the Rules Engine UI without a code change.

Proxy expiry enforcement:

`proxy_authorizations.valid_until` is checked server-side on every vote submission — not just when the proxy is set up.

When a director casts a vote via proxy:

- API checks `proxy_authorizations` WHERE `id = proxy_authorization_id`
- If `valid_until < NOW()`: reject with `PROXY_EXPIRED` error
- UI message: "This proxy authorization expired on [date]. The director [Name] must vote directly, or the admin must upload an updated proxy document."

Proactive warning (cron daily):

- For any open vote window: scan `proxy_authorizations` WHERE `valid_until < NOW() + 2 days`
- Alert admin: "Proxy for [Name] in [requirement] expires in [X] days – verify with the director or upload a renewed PoA before voting closes."

If proxy expires mid-vote-window:

- The proxy vote slot is left unfilled (director must vote directly or be absent)
- Secretary is notified; proxy is flagged as `EXPIRED` in the vote tracker
- The `EXPIRED` proxy record is never deleted – retained for audit purposes

8.4 Conflict of Interest (§7.16(b))

Mandatory declaration before every vote. Recusal is permanent and logged.

8.5 Vendor Decision Record (Immutable)

Written once on final approval; `can_be_modified: false`. Re-decisions create `decision-v2.json`; v1 never touched. Includes: vote summary, recusals, byelaw compliance note, president approval, secretary approval, GitHub commit SHA.

8.6 Post-Selection Tracking

Contract upload; renewal reminder (90 days before expiry); monthly performance rating; complaint log.

9. Module 4 — Financial Tracking

Lightweight tracking module — not a full accounting system. Supports HOTO financial items and ongoing governance.

9.1 What Gets Tracked

- Maintenance collection (per flat, per month, from May 2025)
- Corpus fund (received from builder, interest earned, approved uses)
- Expenses (amount, payee, byelaw authority cited, balance check)
- Builder dues (pending items with SLA dates)

9.2 Corpus Fund Overdraft Prevention

Server-side in every payment approval API:

```
const balance = await supabase.rpc('get_corpus_balance', { p_society_id });
if (balance < amount) {
  return Response.json({ error: 'INSUFFICIENT_BALANCE', current_balance: balance }, { status: 422 });
}
```

This check is **never client-side**. The UI shows the balance for convenience; the API enforces it regardless.

9.3 Corpus Fund APPROVED_USE Approval Chain

`corpus_fund_records` entries with `transaction_type = 'APPROVED_USE'` (withdrawals) are **never created directly** through the financial UI — they are always the output of a completed approval workflow:

Amount	Approval mechanism	Who creates the record
≤ ₹20,000	President unilaterally approves via Expenses module	Record auto-created by API on President's approval click
≤ ₹50,000	Board resolution vote (<code>category = 'FINANCIAL_APPROVAL'</code> in <code>vendor_requirements</code>)	Record auto-created atomically when majority+quorum vote completes
> ₹50,000	Not possible via portal	API blocks with: "Amounts above ₹50,000 require General Body approval (Byelaw §9.11b). This cannot be processed through the portal."

Board financial vote mechanism: For amounts between ₹20,001–₹50,000, the secretary opens a Board resolution vote using the same `vendor_requirements / votes` table structure with `category = 'FINANCIAL_APPROVAL'` and `vendor_id = null`. The motion text (what the money is for, payee, amount) goes in `description`. When quorum (8/14) is reached with a majority yes, the API atomically: (1) writes the `corpus_fund_records` row; (2) runs the overdraft check; (3) creates the `audit_log` entry citing the board resolution. The `board_resolution_ref` on the corpus record stores the `vendor_requirements.id` of the vote.

Treasurer cannot create APPROVED_USE directly. The Treasurer role enters expense records (`expenses` table — running costs) and can enter corpus receipts (`RECEIVED_FROM_BUILDER, INTEREST_EARNED`). Corpus withdrawals always require presidential or board sign-off as above.

9.3 Defaulter Tracking

Threshold	Action	Byelaw
30 days	Reminder email to member	—
60 days	"Defaulting Member" flag; committee notified	§6.36
90 days	Vote rights suspended; <code>payment_status = defaulter_90d</code>	§4.6
90+ days	7-day notice countdown; template auto-populated	§6.37

10. Module 5 — Formal Notice Generation

Integrates with existing letter generation system in the portal.

10.1 Notice Types and Triggers

Notice	Trigger	Reviewer
HOTO 7-day Reminder	Item overdue 7 days	Secretary
HOTO 14-day Escalation	Item overdue 14 days	Secretary
HOTO Legal Notice (auto-draft)	Item overdue 30 days (RERA eligible)	Secretary + President
Snag Rectification Notice	Snag past builder committed date	Secretary
Maintenance Defaulter Notice	90+ days arrears	Secretary
RERA Complaint Package	No response to Legal Notice	Secretary + President

10.2 Auto-Draft Mechanism

At 30 days overdue:

1. System generates draft letter (pre-filled with item details, builder contact, evidence list)
2. Saved to GitHub at `notices/drafts/[date]/[item-id]-draft.pdf`
3. Email to Secretary: "Draft notice ready — [Review] [Send] [Discard]"

Sending updates notice record to `SENT`. No manual letter creation needed.

11. Workflow Engine & Approval Delegation

11.1 Delegation Chain

Default: President + Secretary (both required)

President absent (planned >7 days – §8.2):
Admin sets delegation → VP; all VP actions tagged "per §8.2"

President unexpectedly unavailable (§8.2):
VP may act immediately; flagged for President review on return

Secretary absent (planned – §8.4):
Admin activates Joint Secretary delegation

Both President + VP unavailable:
System shows "Approval chain unavailable"; no approvals possible

11.2 Notification Design

Event	Recipient	Subject	Priority
Pending President approval	President (or VP delegated)	"ACTION REQUIRED: [item] needs your approval"	High
Pending Secretary approval	Secretary (or Joint Sec)	"ACTION REQUIRED: [item] needs your approval"	High
Vote opened	All eligible voters	"VOTE OPEN: [vendor] — closes [date]"	High
Builder SLA overdue 7 days	Committee	"OVERDUE: [item] builder deadline was [date]"	High
Builder SLA overdue 30 days	Secretary + President	"Draft formal notice ready for [item]"	High
GitHub health check failed	Secretary + admin	"URGENT: Governance storage unavailable"	Critical
Weekly digest	All committee	"HOTO Week [N] Summary"	Low

12. Non-Tech User Experience Specification

12.1 Design Rules for Non-Tech Screens

- "My Actions" is the home screen — not the full dashboard
- Maximum 2 action buttons per item
- Status = one word + one color: Approved (green), Pending (orange), Rejected (red)
- Minimum 56px tap targets for primary actions
- Minimum 16px text
- No icons without text labels
- All text in plain English, no abbreviations

12.2 My Actions Screen

Good morning, Bal Reddy

Tuesday, June 3, 2026

YOU NEED TO DO THESE NOW

 APPROVE

Lift No. 2 AMC Transfer (Block B)

Secretary approved this on June 2.

[Read the Details]

[APPROVE]

 Nothing else needs your attention today

[View All HOTO Items]

[View All Snags]

12.3 Approval Screen

← My Actions

Lift No. 2 AMC Transfer (Block B)

What is this about?

Transfer the KONE lift service contract from Ankura Homes to our association. Secretary has approved. Your approval completes this.

Documents attached (3):

 KONE AMC Contract [View]

 NOC from Builder [View]

 New AMC Agreement [View]

 APPROVE

I confirm this is correct

 NOT YET

I have questions or concerns

"NOT YET" opens a text field; comment logged; Secretary notified. Item stays at PENDING_PRESIDENT.

12.4 Pre-Launch Walkthrough Protocol

Before go-live (May 30): Secretary walks through with Bal Reddy using a test HOTO item. Bal Reddy performs a real approval and a real vote. Any confusion fixed before May 31 go-live.

12.5 Validation Messages & Blocked Feature UX

Core principle for every blocked action, validation failure, or permission gate:

1. **What** was blocked — in plain English, never a raw error code

2. **Why** it was blocked — the actual reason; include byelaw citation after plain explanation
3. **What to do next** — specific and actionable; name the person or step

No user should ever see "Error 403", "Forbidden", or "An error occurred. Please try again." These are developer messages, not user messages.

Display Patterns

Pattern	When to use	Position
Inline error	Form field validation failure	Below the field, red text
Toast (success)	Background operation completed	Top-right, auto-dismiss 4s
Toast (error)	Background operation failed	Top-right, persists until dismissed
Contextual info box	Permission gate within a page	Inline with the blocked action
Page-level banner	System-wide issue (GitHub down, approval chain broken)	Sticky top, amber or red
Blocking modal	Irreversible action (approve, vote, deactivate, bulk election)	Center screen, must confirm

Message Catalog — Byelaw-Driven Blocks

Scenario	User-Facing Message
Vote suspended — maintenance arrears	"Your voting rights are currently suspended because maintenance for Flat [N] is [X] days overdue (Byelaw §4.6). Please contact the Secretary to arrange payment. Once settled, your voting access is restored automatically."
Quorum not yet met	"Voting is open, but [X] more directors need to vote before results can be counted. Required: 8 of 14 · Voted so far: [Y]"
Financial limit — Secretary	"Your approval authority is up to ₹10,000 (Byelaw §9.11a). This expense is ₹[amount]. It needs the President's approval. Please forward the details to the President."
Financial limit — President	"This expense (₹[amount]) exceeds your authority of ₹20,000 (Byelaw §9.11a). Amounts up to ₹50,000 require a Board resolution. Contact the Secretary to open a Board vote."
Financial limit — Board	"This expense (₹[amount]) exceeds the Board's authority of ₹50,000 (Byelaw §9.11b). It requires General Body approval, which cannot be processed through this portal. Raise it at the next Annual General Body Meeting."
Corpus fund insufficient	"The corpus fund currently holds ₹[balance]. This withdrawal (₹[amount]) would exceed the available balance. Please reduce the amount or wait until additional funds are received."
Cash payment attempted	"Cash payments are not permitted for this society (Byelaw §5.3p and §9.1). Please select an electronic payment mode — bank transfer, UPI, or cheque."

Scenario	User-Facing Message
President and VP both absent	Page banner: "Approval chain unavailable — the President and Vice President are currently unavailable. No HOTO or vendor approvals can be completed until this is resolved. Contact Admin to activate delegation."
Approval chain incomplete (dual sign-off)	"This item needs both President and Secretary approval (Byelaw §8.1 and §8.3). Secretary approved on [date]. Waiting for the President. No action needed from you right now — the President has been notified."
Required documents missing	"This item cannot move to review until all required documents are uploaded. Missing: [list]. The Secretary or President can override this gate if there is a valid reason."
Director conflict of interest	"You declared a conflict of interest for this evaluation. You cannot cast a vote on this requirement (Byelaw §7.16b). Your recusal has been permanently recorded."

Message Catalog — Feature Permission Gates

Scenario	UX behaviour
Feature not in user's role	The button or section is hidden entirely. No error shown — users see only what they can do.
Locked feature (byelaw-mandated)	 icon shows. Hover tooltip: "This permission is required by the society's byelaws and cannot be changed."
Admin-only action	"This action can only be performed by the system admin. Contact [admin name] if you need this done."
Approval gate — wrong role	"This approval requires the President's confirmation. You can read the item but cannot approve it. The President has been notified."
Temporarily restricted feature (custom override)	"Access to [feature] is currently restricted for your account. Contact the Admin if you believe this is incorrect."

Message Catalog — Background Operations

Scenario	User-Facing Message
File upload queued	Toast: "Your file is being saved securely. You'll receive an email when it's ready. (Usually under 1 minute)"
File upload processing	Status badge next to file: "Saving... [spinner]" → auto-updates to "Saved ✓"
File upload failed (all retries exhausted)	Toast (persistent): "Your file could not be saved after multiple attempts. Please try uploading again. If the problem continues, contact admin."
PDF generation started	Spinner with text: "Generating your letter... This usually takes 30–60 seconds."
PDF generation complete	Spinner replaced by: [Download Letter ↓] button — no page refresh needed

Scenario	User-Facing Message
PDF generation failed	"The letter could not be generated. Please try again. If the issue continues, contact admin."
GitHub storage unavailable	Page banner (amber): "Document storage is temporarily unavailable. You can still view and manage items, but file uploads will queue until storage is restored. The admin has been notified automatically."

Message Catalog — Auth & Session

Scenario	User-Facing Message
Session expired	Full page: "Your session has expired for security. [Log In Again]" — auto-redirects in 5 seconds
Account deactivated	"Your portal access has been deactivated. If you believe this is an error, contact the Secretary or Admin."
Privacy consent required (first login)	Full-screen modal — cannot be dismissed until accepted. Body: "Before you access the portal, we need your consent to store and use your contact information for UTA MACS governance communications. This is required by the Digital Personal Data Protection Act, 2023."
Invite link expired	"This invitation has expired (invitations are valid for 7 days). Contact the admin to request a new invitation."
Invite link already used	"This invitation has already been used. If you registered, [Log In here]. If you did not register, contact the admin — your invitation may have been used by someone else."

UX Rules for Non-Tech Users (My Actions Screen)

For Bal Reddy and the Working President, **no action should ever dead-end** — every blocked state must tell them what happens next without requiring them to take any action themselves:

If a blocked item appears on My Actions, show:

WAITING – not ready for you yet

[Item title]

[Plain English explanation of what is waiting]

No action needed from you right now.

You'll be notified when this is ready.

Never show: Technical details, error codes, "Something went wrong", or blank/empty states without explanation.

For voting specifically (non-tech user):

**VOTE NEEDED**

Choose a Property Management Company

Read about each option below, then tap your choice. Your vote is final.

[MyGate – ₹45/flat/month]

★ 4.2/5 by committee Site visit: ✓

[Choose MyGate]

[NoBroker – ₹38/flat/month]

★ 3.8/5 by committee Site visit: ✓

[Choose NoBroker]

[I am not voting on this (explain why)]

5 of 14 directors have voted.

Voting closes: Friday, 5 June at 6:00 PM

After voting, immediately show:



Your vote for [MyGate] has been recorded.

Thank you. You'll hear the outcome when voting closes.

My Actions — overflow/pagination:

- Maximum 5 items shown on My Actions at once; excess items shown as: "5 more items need your attention → [View All]"
- Items sorted by urgency: (1) approvals pending >48h, (2) votes closing within 24h, (3) everything else by date
- Empty state: "Nothing needs your attention today. Check back tomorrow."

Implementation Note: No Visible 403 Pages

If a user navigates directly to a URL for a feature their role doesn't include (e.g., `/portal/admin/elections` without `is_admin`):

→ Server-side: redirect to `/portal/dashboard` with a query param: `?blocked=insufficient_role`

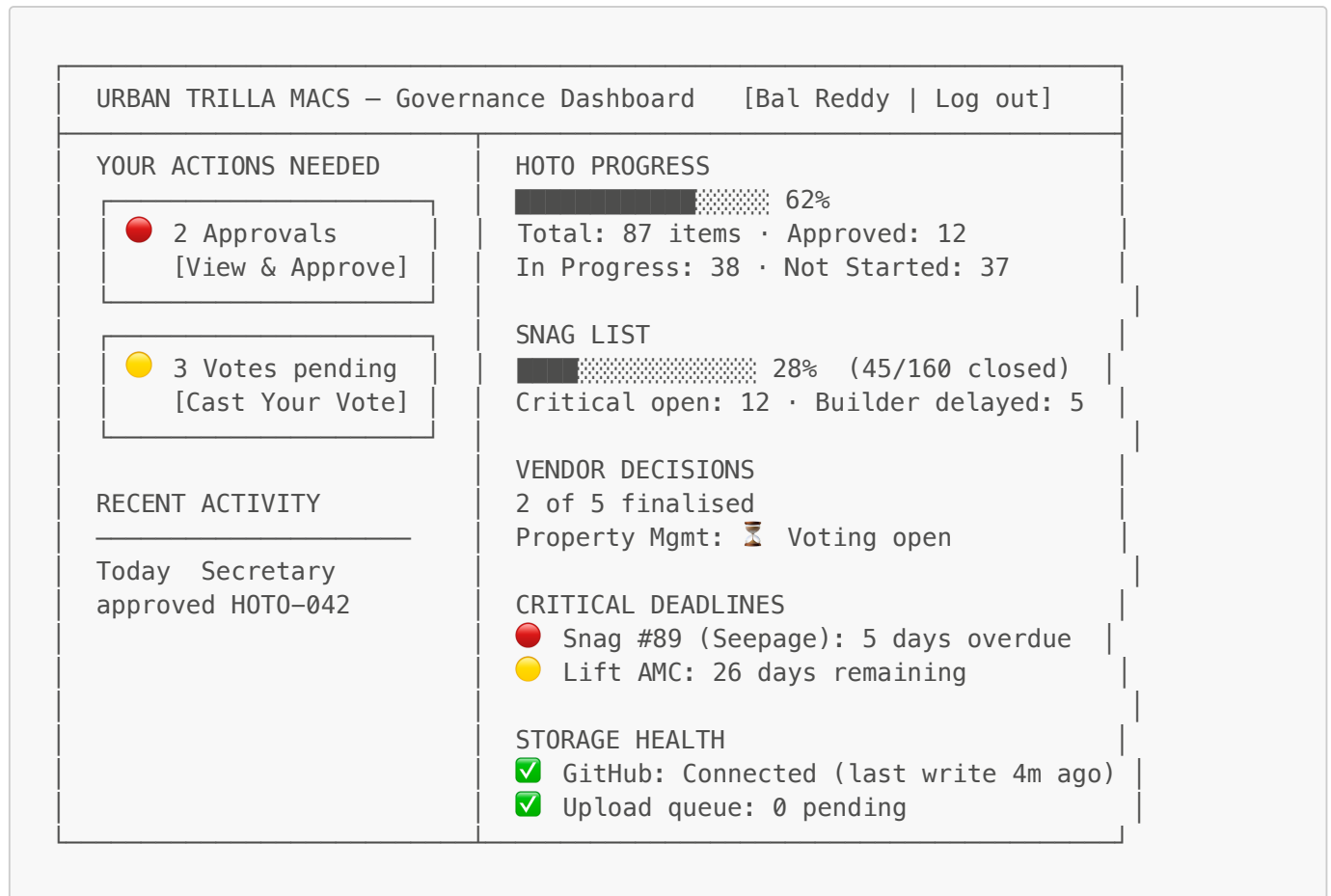
→ Dashboard: shows a dismissable notice:

"You don't have access to that page. If you need access, contact the admin."

NOT: a bare "403 Forbidden" or "Access Denied" page.

13. Dashboard & UX Design

13.1 Full Dashboard Layout



13.2 Mobile Design

- Responsive; works in any browser without app install
- "My Actions" collapses to top on mobile
- Camera button on document upload
- Large tap targets (min 56px)
- Bookmarkable at portal.utamacs.org/portal/my-actions for non-tech users

14. Git Storage Strategy

14.1 Repository Structure

```

utamacs/governance-data/
├── README.md
├── RUNBOOK.md
├── _meta/
│   ├── committee-roster.json
│   ├── voting-policy.md           # Committed before first vote
│   └── scope-v1.md               # Committed before launch
├── hoto/
│   ├── _index.json
│   └── [Category]/[HOTO-NNN]/
│       ├── item.json
│       ├── comments.json         # Append-only
│       ├── approvals.json       # Append-only
│       └── documents/

```

```

├── snags/
│   ├── _index.json
│   ├── Common-Area/
│   └── Individual-Apartment/
├── vendors/
│   ├── _index.json
│   └── [REQ-MNN]/
│       ├── votes.json           # Append-only
│       ├── decision.json       # Written ONCE; immutable
│       └── proxy-authorizations/
├── notices/
│   ├── drafts/
│   ├── builder/
│   └── members/
├── finances/
│   ├── maintenance/
│   ├── corpus/
│   └── expenses/
├── audit/
│   ├── [YYYY-MM]/
│   └── [YYYY-MM-DD].jsonl      # Append-only

```

14.2 Commit Convention

```

create(HOTO-042): Lift No.2 AMC Transfer – Treasurer
upload(HOTO-042): kone-amc.pdf [sha:abc123] source="Ankura Homes 2026-04-15" – Treasurer
status(HOTO-042): IN_PROGRESS → EVIDENCE_UPLOADED – Treasurer
approve(HOTO-042): President approval [per §8.1] – Bal Reddy
approve(HOTO-042): Secretary approval → APPROVED
vote(REQ-001): ApartmentAdda – Secretary [8/14 votes]
decide(REQ-001): ApartmentAdda selected (12/14 quorum per §7.16)
snag-create(SNAG-089): Block B seepage [COMMON_AREA] – Ravi
role-change: executive → secretary for [Name] – President [election-2026-06-15]
health-check: OK [latency:234ms]

```

14.3 Immutability Rules

- **decision.json**: written once; re-decisions create **decision-v2.json**
- **approvals.json**, **votes.json**, **audit/*.jsonl**: append-only, never modified
- **comments.json**: edits add new entry with **edited_at**; original preserved

15. Data Model

15.1 Complete Schema

```

--
-- Societies (top-level multi-tenancy anchor; all other tables FK here)
-- UTA MACS has exactly one society. This enables future multi-society
-- hosting without schema changes to any other table.
--
CREATE TABLE IF NOT EXISTS societies (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name TEXT NOT NULL,
registration_number TEXT UNIQUE NOT NULL,
address TEXT,
district TEXT,
state TEXT DEFAULT 'Telangana',
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Seed: UTA MACS (use fixed UUID so all env vars and seeds reference it
consistently)
INSERT INTO societies (id, name, registration_number, address, district)
VALUES (
    '00000000-0000-0000-0000-000000000001',
    'Urban Trilla Apartment Owners Mutually Aided Cooperative Maintenance Society
    Limited',
    'TG/RRD/MACS/2026-15/FOW & M',
    'SY NO:425/2/1, Kondakal Village, Shankarpally Mandal',
    'Rangareddy'
) ON CONFLICT DO NOTHING;

-- -----
-- EXISTING BUG FIX: user_roles table missing unique constraint
-- This causes role changes via UI to silently fail or create duplicates
-- Run this migration first (migration 027)
-- -----
ALTER TABLE user_roles
ADD CONSTRAINT IF NOT EXISTS user_roles_user_society_unique
UNIQUE (user_id, society_id);

-- -----
-- Profiles (existing table – extend)
-- -----
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS
portal_role TEXT DEFAULT 'executive',
-- FOUR values only: 'member' | 'executive' | 'secretary' | 'president'
-- This controls ALL permission and approval logic. No other role values exist.
-- Old roles (vice_president, joint_secretary, treasurer, etc.) are removed –
see migration below.
committee_title TEXT,
-- Display label ONLY – no effect on permissions, approval authority, or any
system logic.
-- Examples: 'President', 'Vice President', 'Working President', 'General
Secretary',
-- 'Joint Secretary', 'Treasurer', 'Joint Treasurer', 'Executive
Member', null
-- Set by admin when inviting or changing a member's designation.
is_admin BOOLEAN DEFAULT false,
-- Orthogonal to portal_role. Admin executes user management. Having is_admin
does NOT
-- grant governance powers (voting, HOTO approvals) – those come from
portal_role only.
payment_status TEXT DEFAULT 'current',
last_maintenance_paid_date DATE,
maintenance_arrears_days INTEGER DEFAULT 0,
privacy_consent_given BOOLEAN DEFAULT false,
privacy_consent_at TIMESTAMPTZ,
is_active BOOLEAN DEFAULT true,

```

```

email_digest_enabled BOOLEAN DEFAULT true;

-- -----
-- DATA MIGRATION: Collapse 9 old roles → 4 new roles
-- Run AFTER adding committee_title column; sets both portal_role and
committee_title
-- from the previous role values. Safe to run multiple times (idempotent).
-- -----
-- Step 1: Set committee_title from old portal_role values (before collapsing
roles)
UPDATE profiles SET committee_title =
CASE portal_role
WHEN 'president' THEN 'President'
WHEN 'vice_president' THEN 'Vice President'
WHEN 'working_president' THEN 'Working President'
WHEN 'secretary' THEN 'General Secretary'
WHEN 'joint_secretary' THEN 'Joint Secretary'
WHEN 'treasurer' THEN 'Treasurer'
WHEN 'joint_treasurer' THEN 'Joint Treasurer'
WHEN 'executive' THEN 'Executive Member'
WHEN 'member' THEN NULL
ELSE committee_title -- leave unchanged if already set
END
WHERE committee_title IS NULL;

-- Step 2: Collapse old role values → 4-role model
UPDATE profiles SET portal_role =
CASE portal_role
WHEN 'working_president' THEN 'executive'
WHEN 'vice_president' THEN 'executive'
WHEN 'joint_secretary' THEN 'executive'
WHEN 'treasurer' THEN 'executive'
WHEN 'joint_treasurer' THEN 'executive'
ELSE portal_role -- 'member', 'executive', 'secretary', 'president' unchanged
END
WHERE portal_role NOT IN ('member', 'executive', 'secretary', 'president');

-- Step 3 (manual, post-migration): Grant finance feature overrides to
Treasurer/Joint Treasurer
-- These are user-specific overrides – run in the application code or manually:
-- For each profile WHERE committee_title IN ('Treasurer', 'Joint Treasurer'):
-- INSERT INTO user_feature_overrides (society_id, user_id, feature, enabled,
reason, granted_by)
-- VALUES (society_id, user_id, 'finance.view', true, 'Treasurer designation',
admin_id),
-- (society_id, user_id, 'finance.enter', true, 'Treasurer designation',
admin_id)
-- ON CONFLICT DO NOTHING;

-- -----
-- Privacy Consents (DPDP Act compliance)
-- -----
CREATE TABLE privacy_consents (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID REFERENCES profiles NOT NULL,
policy_version TEXT NOT NULL,
consent_given BOOLEAN NOT NULL,
consent_at TIMESTAMPTZ DEFAULT NOW(),

```



```

    ip_hash TEXT,
    user_agent_hash TEXT
);

-- -----
-- Member Invites (invite-only registration)
-- -----

CREATE TABLE member_invites (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    email TEXT NOT NULL,
    flat_number TEXT,
    intended_role TEXT NOT NULL DEFAULT 'member',
    invited_by UUID REFERENCES profiles NOT NULL,
    token TEXT UNIQUE NOT NULL DEFAULT encode(gen_random_bytes(32), 'hex'),
    token_expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '7 days',
    accepted BOOLEAN DEFAULT false,
    accepted_at TIMESTAMPTZ,
    accepted_user_id UUID REFERENCES profiles,
    cancelled BOOLEAN DEFAULT false,
    cancelled_by UUID REFERENCES profiles,
    cancelled_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_member_invites_token ON member_invites(token)
    WHERE NOT accepted AND NOT cancelled;
-- Token lookup uses constant-time comparison (Node.js crypto.timingSafeEqual) to
-- prevent timing attacks. The raw token value is never written to application
-- logs.
-- Token is invalidated (accepted = true) atomically in the same DB transaction
-- that creates the profiles row – no window exists where a token remains valid
-- after the account is created.

-- -----
-- Role Change Log (fast role history; complements audit_log)
-- -----

CREATE TABLE role_change_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    user_id UUID REFERENCES profiles NOT NULL,
    old_role TEXT NOT NULL,
    new_role TEXT NOT NULL,
    old_title TEXT,           -- committee_title before change (null if no title set)
    new_title TEXT,          -- committee_title after change (null if cleared)
    changed_by UUID REFERENCES profiles NOT NULL, -- always the admin
    reason TEXT NOT NULL,    -- free-text reason for audit
    trail
    change_type TEXT NOT NULL DEFAULT 'ROLE_AND_TITLE',
    -- Values: 'ROLE_AND_TITLE' | 'ROLE_ONLY' | 'TITLE_ONLY' | 'ELECTION'
    election_event_id UUID,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_role_change_user ON role_change_log(user_id, created_at DESC);

-- -----
-- Election Events (groups bulk role changes)
-- -----

CREATE TABLE election_events (

```

```

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    election_date DATE NOT NULL,
    description TEXT NOT NULL,
    total_role_changes INTEGER DEFAULT 0,
    outcome_document_id TEXT REFERENCES documents, -- optional: uploaded AGM
minutes
    created_by UUID REFERENCES profiles, -- the admin who executed this
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- -----
-- Feature Permissions (which features each role can access)
-- Only admin (is_admin=true) can change these
-- -----

CREATE TABLE feature_permissions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    role TEXT NOT NULL,
    feature TEXT NOT NULL,
    enabled BOOLEAN NOT NULL DEFAULT true,
    is_locked BOOLEAN NOT NULL DEFAULT false, -- locked = byelaw-mandated; cannot
be toggled
    last_changed_by UUID REFERENCES profiles, -- always the admin
    last_changed_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE (society_id, role, feature)
);

-- -----
-- User Feature Overrides (per-user exceptions to role defaults)
-- Only admin (is_admin=true) can grant these
-- -----

CREATE TABLE user_feature_overrides (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    user_id UUID REFERENCES profiles NOT NULL,
    feature TEXT NOT NULL,
    enabled BOOLEAN NOT NULL,
    reason TEXT NOT NULL, -- why this override is needed
(audit trail)
    granted_by UUID REFERENCES profiles NOT NULL, -- always the admin
    granted_at TIMESTAMPTZ DEFAULT NOW(),
    expires_at TIMESTAMPTZ,
    revoked_at TIMESTAMPTZ,
    revoked_by UUID REFERENCES profiles,
    UNIQUE (society_id, user_id, feature)
);
CREATE INDEX idx_user_feature_overrides_user ON user_feature_overrides(user_id)
WHERE revoked_at IS NULL;

-- -----
-- Upload Queue
-- -----

CREATE TABLE upload_queue (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    uploaded_by UUID REFERENCES profiles,
    item_type TEXT NOT NULL,

```

```

    item_id TEXT NOT NULL,
    file_name TEXT NOT NULL,
    file_size_bytes INTEGER,
    file_type TEXT,
    file_hash_sha256 TEXT,
    source_description TEXT,
    target_github_path TEXT NOT NULL,
    -- Server-side validation required before INSERT:
    --   Must match: ^(hoto|snags|vendors|notices|finances|audit)/[a-zA-Z0-9/_.-]+
    --   Must not contain: '..', '//' , or null bytes
    --   Rejection response: { error: 'INVALID_PATH', message: 'File path is not
permitted.' }
    status TEXT DEFAULT 'PENDING',
    attempts INTEGER DEFAULT 0,
    last_attempt_at TIMESTAMPTZ,
    error_message TEXT,
    github_sha TEXT,
    document_id TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_upload_queue_status ON upload_queue(status);

-- -----
-- GitHub API Log
-- -----
CREATE TABLE github_api_log (
    id BIGSERIAL PRIMARY KEY,
    operation TEXT NOT NULL,
    success BOOLEAN NOT NULL,
    latency_ms INTEGER,
    error_message TEXT,
    github_path TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_github_api_log_created ON github_api_log(created_at DESC);

-- -----
-- PDF Generation Jobs
-- -----
CREATE TABLE pdf_generation_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    requested_by UUID REFERENCES profiles,
    job_type TEXT NOT NULL,
    letter_id TEXT,
    template TEXT,
    input_data JSONB,
    status TEXT DEFAULT 'QUEUED',
    attempts INTEGER DEFAULT 0,
    github_path TEXT,
    error_message TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    completed_at TIMESTAMPTZ,
    purged_at TIMESTAMPTZ -- set when input_data PII is scrubbed (30 days post-
completion)
);
-- Retention policy: rows are never deleted (job history = audit record).
-- 30 days after reaching DONE or FAILED, a daily cron sets input_data = null and

```

```
-- purged_at = NOW(). This removes any PII embedded in the letter template input
-- while preserving the job existence, type, and outcome for audit purposes.
```

```
-- -----
-- HOTO Items
-- -----
```

```
CREATE TABLE hoto_items (
  id TEXT PRIMARY KEY,
  society_id UUID NOT NULL,
  ascenza_category TEXT NOT NULL,
  title TEXT NOT NULL,
  description TEXT,
  builder_commitment TEXT,
  builder_contact TEXT,
  priority TEXT DEFAULT 'MEDIUM',
  status TEXT NOT NULL DEFAULT 'NOT_STARTED',
  deadline DATE,
  builder_sla_date DATE,
  days_overdue INTEGER DEFAULT 0,
  responsible_role TEXT,
  responsible_user_id UUID REFERENCES profiles,
  rera_escalation_eligible BOOLEAN DEFAULT false,
  notice_sent BOOLEAN DEFAULT false,
  notice_sent_date TIMESTAMPTZ,
  notice_draft_path TEXT,
  dependencies TEXT[],
  president_approved_at TIMESTAMPTZ,
  president_approved_by UUID REFERENCES profiles,
  secretary_approved_at TIMESTAMPTZ,
  secretary_approved_by UUID REFERENCES profiles,
  governance_notes TEXT,
  created_by UUID REFERENCES profiles,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  last_updated_at TIMESTAMPTZ DEFAULT NOW(),
  github_path TEXT
);
```

```
-- -----
-- HOTO Required Documents
-- -----
```

```
CREATE TABLE hoto_required_docs(
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  hoto_item_id TEXT REFERENCES hoto_items,
  doc_name TEXT NOT NULL,
  required BOOLEAN DEFAULT true,
  uploaded BOOLEAN DEFAULT false,
  document_id TEXT,
  bypass_by UUID REFERENCES profiles,
  bypass_reason TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- -----
-- Snag Items
-- -----
```

```
CREATE TABLE snag_items (
  id TEXT PRIMARY KEY,
  society_id UUID NOT NULL,
```

```

snag_scope TEXT NOT NULL DEFAULT 'COMMON_AREA',
category TEXT NOT NULL,
subcategory TEXT,
location TEXT NOT NULL,
flat_number TEXT,
description TEXT NOT NULL,
severity TEXT DEFAULT 'MEDIUM',
status TEXT DEFAULT 'OPEN',
ascenza_reference TEXT,
builder_committed_date DATE,
builder_sla_days_overdue INTEGER DEFAULT 0,
notice_sent BOOLEAN DEFAULT false,
formal_notice_id TEXT,
video_url TEXT,
reported_by UUID REFERENCES profiles,
reported_date DATE DEFAULT CURRENT_DATE,
verified_by UUID REFERENCES profiles,
verified_at TIMESTAMPTZ,
responsible_role TEXT,          -- for auto-reassignment on role change
($5.8)
responsible_user_id UUID REFERENCES profiles,
reopen_reason TEXT,             -- required when VERIFIED_CLOSED → REOPENED
deleted BOOLEAN DEFAULT false,
deleted_by UUID REFERENCES profiles,
deleted_at TIMESTAMPTZ,
deletion_reason TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
github_path TEXT
);

-- -----
-- Documents
-- -----

CREATE TABLE documents (
  id TEXT PRIMARY KEY,
  item_type TEXT NOT NULL,
  item_id TEXT NOT NULL,
  name TEXT NOT NULL,
  file_type TEXT,
  file_size_bytes INTEGER,
  file_hash_sha256 TEXT NOT NULL,
  source_description TEXT,
  github_path TEXT NOT NULL,
  github_sha TEXT,
  upload_queue_id UUID REFERENCES upload_queue,
  uploaded_by UUID REFERENCES profiles,
  uploaded_at TIMESTAMPTZ DEFAULT NOW(),
  description TEXT,
  is_confidential BOOLEAN DEFAULT false,
  superseded_by TEXT REFERENCES documents,
  superseded_at TIMESTAMPTZ
);

-- -----
-- Vendor Requirements
-- -----

CREATE TABLE vendor_requirements (
  id TEXT PRIMARY KEY,

```

```
society_id UUID NOT NULL,
category TEXT NOT NULL,
title TEXT NOT NULL,
description TEXT,
status TEXT DEFAULT 'DRAFT',
voting_opens_at TIMESTAMPTZ,
voting_closes_at TIMESTAMPTZ,
quorum_required INTEGER DEFAULT 8,
selected_vendor_id TEXT,
voting_policy_committed BOOLEAN DEFAULT false,
created_by UUID REFERENCES profiles,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Vendors
```

```
CREATE TABLE vendors (
  id TEXT PRIMARY KEY,
  requirement_id TEXT REFERENCES vendor_requirements,
  vendor_name TEXT NOT NULL,
  contact_person TEXT,
  contact_email TEXT,
  contact_phone TEXT,
  site_visited BOOLEAN DEFAULT false,
  quote_monthly NUMERIC(12,2),
  quote_setup NUMERIC(12,2),
  submitted_at TIMESTAMPTZ,
  contract_start_date DATE,
  contract_end_date DATE,
  renewal_reminder_sent BOOLEAN DEFAULT false,
  github_path TEXT
);
```

```
-- Proxy Authorizations
```

```
CREATE TABLE proxy_authorizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  principal_user_id UUID REFERENCES profiles NOT NULL,
  proxy_user_id UUID REFERENCES profiles NOT NULL,
  requirement_id TEXT REFERENCES vendor_requirements,
  proxy_document_id TEXT REFERENCES documents,
  valid_from DATE NOT NULL,
  valid_until DATE,
  activated_by UUID REFERENCES profiles,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Votes
```

```
CREATE TABLE votes (
  id TEXT PRIMARY KEY,
  requirement_id TEXT REFERENCES vendor_requirements,
  voter_id UUID REFERENCES profiles,
  proxy_authorization_id UUID REFERENCES proxy_authorizations,
  vendor_id TEXT REFERENCES vendors,
```

```

    reason TEXT NOT NULL,
    conflict_declared BOOLEAN DEFAULT false,
    recused BOOLEAN DEFAULT false,
    cast_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(requirement_id, voter_id)
);

-- -----
-- Maintenance Records
-- -----

CREATE TABLE maintenance_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    flat_number TEXT NOT NULL,
    member_id UUID REFERENCES profiles,
    amount NUMERIC(10,2) NOT NULL,
    period_month INTEGER NOT NULL,
    period_year INTEGER NOT NULL,
    paid_date DATE,
    payment_mode TEXT,
    reference_number TEXT,
    recorded_by UUID REFERENCES profiles,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- -----
-- Corpus Fund Records
-- -----

CREATE TABLE corpus_fund_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    transaction_type TEXT NOT NULL,
    amount NUMERIC(12,2) NOT NULL,
    description TEXT,
    date DATE NOT NULL,
    approved_by UUID REFERENCES profiles,
    board_resolution_ref TEXT,
    payment_mode TEXT,
    reference_number TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION get_corpus_balance(p_society_id UUID)
RETURNS NUMERIC AS $$
    SELECT COALESCE(
        SUM(CASE WHEN transaction_type IN ('RECEIVED_FROM_BUILDER', 'INTEREST_EARNED')
            WHEN transaction_type = 'APPROVED_USE' THEN -amount END), 0)
    FROM corpus_fund_records WHERE society_id = p_society_id;
$$ LANGUAGE SQL STABLE SECURITY DEFINER;

-- -----
-- Expenses
-- -----

CREATE TABLE expenses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID NOT NULL,
    amount NUMERIC(12,2) NOT NULL,

```

```

payee TEXT NOT NULL,
purpose TEXT NOT NULL,
expense_date DATE NOT NULL,
payment_mode TEXT NOT NULL,
reference_number TEXT,
is_recurring BOOLEAN DEFAULT false,
sanctioned_by_role TEXT,
sanctioned_by UUID REFERENCES profiles,
byelaw_authority TEXT,
board_resolution_ref TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

--
-- Comments
--

CREATE TABLE comments (
  id TEXT PRIMARY KEY,
  item_type TEXT NOT NULL,
  item_id TEXT NOT NULL,
  parent_comment_id TEXT REFERENCES comments,
  author_id UUID REFERENCES profiles,
  content TEXT NOT NULL,
  is_pinned BOOLEAN DEFAULT false,
  edited_at TIMESTAMPTZ,
  edited_content TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  github_commit TEXT
  -- No deleted column: comments are permanent audit records
);

--
-- Formal Notices
--

CREATE TABLE notices (
  id TEXT PRIMARY KEY,
  notice_type TEXT NOT NULL,
  recipient TEXT NOT NULL,
  recipient_type TEXT NOT NULL,
  related_item_type TEXT,
  related_item_id TEXT,
  auto_generated BOOLEAN DEFAULT false,
  status TEXT DEFAULT 'DRAFT',
  sent_date DATE,
  sent_by UUID REFERENCES profiles,
  document_path TEXT,
  response_received BOOLEAN DEFAULT false,
  response_date DATE,
  rera_filed BOOLEAN DEFAULT false,
  rera_date DATE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

--
-- Approval Delegations
--

CREATE TABLE approval_delegations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```



```

society_id UUID NOT NULL,
from_role TEXT NOT NULL,
to_user_id UUID REFERENCES profiles,
reason TEXT NOT NULL,
delegation_type TEXT NOT NULL,
active BOOLEAN DEFAULT true,
activated_by UUID REFERENCES profiles,
activated_at TIMESTAMPTZ DEFAULT NOW(),
deactivated_at TIMESTAMPTZ,
notes TEXT
);

-- -----
-- System Config (key-value store for runtime flags like circuit_breaker)
-- -----

CREATE TABLE system_config (
  key TEXT PRIMARY KEY,
  value JSONB NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  updated_by UUID REFERENCES profiles
);
INSERT INTO system_config (key, value) VALUES
  ('github_circuit_breaker', '"CLOSED"'),
  ('github_consecutive_failures', '0')
ON CONFLICT DO NOTHING;

-- -----
-- Byelaw / Business Rules Engine (see §23)
-- All configurable parameters – byelaw-locked or operational
-- -----

CREATE TABLE rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  society_id UUID REFERENCES societies NOT NULL,
  rule_category TEXT NOT NULL, -- 'PARAMETER', 'APPROVAL', 'ESCALATION',
'NOTIFICATION', 'VALIDATION'
  rule_code TEXT NOT NULL,
  label TEXT NOT NULL,
  description TEXT,
  byelaw_reference TEXT,
  value_type TEXT NOT NULL,
  current_value JSONB NOT NULL,
  default_value JSONB NOT NULL,
  is_locked BOOLEAN DEFAULT true,
  effective_from DATE NOT NULL DEFAULT CURRENT_DATE,
  changed_by UUID REFERENCES profiles,
  changed_at TIMESTAMPTZ DEFAULT NOW(),
  change_reason TEXT,
  UNIQUE (society_id, rule_code)
);
CREATE INDEX idx_rules_society_category ON rules(society_id, rule_category,
rule_code);

-- -----
-- Cron Heartbeats (§4.5 – silence detection)
-- -----

CREATE TABLE cron_heartbeats (
  id BIGSERIAL PRIMARY KEY,
  cron_name TEXT NOT NULL,

```

```

    run_at TIMESTAMPTZ DEFAULT NOW(),
    status TEXT NOT NULL,          -- 'OK', 'PARTIAL', 'FAILED', 'CIRCUIT_OPEN'
    items_processed INTEGER DEFAULT 0,
    items_failed INTEGER DEFAULT 0,
    duration_ms INTEGER,
    error_message TEXT
);
CREATE INDEX idx_cron_heartbeats_name ON cron_heartbeats(cron_name, run_at DESC);

-- -----
-- Cron Locks (§4.5 – idempotency for Vercel Cron duplicates)
-- -----

CREATE TABLE cron_locks (
    item_type TEXT NOT NULL,
    item_id TEXT NOT NULL,
    run_id UUID NOT NULL,
    acquired_at TIMESTAMPTZ DEFAULT NOW(),
    expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '10 minutes',
    PRIMARY KEY (item_type, item_id)
);

-- -----
-- Email Drafts (§24 – pre-generated formal communications)
-- -----

CREATE TABLE email_drafts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    society_id UUID REFERENCES societies NOT NULL,
    tier INTEGER NOT NULL CHECK (tier IN (1, 2, 3)),
    triggered_by TEXT NOT NULL,
    trigger_resource_type TEXT,
    trigger_resource_id TEXT,
    recipient_type TEXT NOT NULL,
    recipient_email TEXT,
    recipient_name TEXT,
    subject TEXT NOT NULL,
    body_html TEXT NOT NULL,
    body_text TEXT NOT NULL,
    suggested_sender_name TEXT NOT NULL,
    suggested_sender_email TEXT NOT NULL,
    status TEXT DEFAULT 'DRAFT' CHECK (status IN
('DRAFT', 'REVIEWED', 'SENT', 'DISCARDED')),
    reviewed_by UUID REFERENCES profiles,
    reviewed_at TIMESTAMPTZ,
    sent_by UUID REFERENCES profiles,
    sent_at TIMESTAMPTZ,
    resend_message_id TEXT,
    discarded_by UUID REFERENCES profiles,
    discarded_reason TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_email_drafts_status ON email_drafts(society_id, status,
created_at DESC);

-- -----
-- upload_queue: add backoff support (§4.5)
-- -----

ALTER TABLE upload_queue
    ADD COLUMN IF NOT EXISTS backoff_until TIMESTAMPTZ,

```

```

ADD COLUMN IF NOT EXISTS idempotency_key TEXT UNIQUE;

-- -----
-- profiles: email digest opt-out ($26.6 gap #4)
-- -----
ALTER TABLE profiles
  ADD COLUMN IF NOT EXISTS email_digest_enabled BOOLEAN DEFAULT true;

-- -----
-- Audit Log
-- -----
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  society_id UUID NOT NULL,
  actor_id UUID REFERENCES profiles,
  action TEXT NOT NULL,
  resource_type TEXT NOT NULL,
  resource_id TEXT NOT NULL,
  old_values JSONB,
  new_values JSONB,
  byelaw_reference TEXT,
  ip_hash TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_audit_log_resource ON audit_log(resource_type, resource_id);
CREATE INDEX idx_audit_log_actor ON audit_log(actor_id, created_at DESC);

```

16. Security & Privacy Compliance

16.1 GitHub App Private Key Security

1. Key lives **only** in Vercel environment variables (`GITHUB_APP_PRIVATE_KEY`)
2. Never in code, `.env` files, or any git-tracked file
3. Pre-commit hook on `utamacs-website` repo:

```

if git diff --cached --name-only | xargs grep -l "BEGIN RSA PRIVATE
KEY\|BEGIN EC PRIVATE KEY" 2>/dev/null; then
  echo "ERROR: Private key in staged files. Commit blocked."; exit 1
fi

```

4. Quarterly rotation; procedure in `RUNBOOK.md`
5. GitHub App permissions: `contents:write` on `governance-data` only

16.2 Row-Level Security

All sensitive tables have RLS enabled. `anon` key has zero access.

```

ALTER TABLE hoto_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE maintenance_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE votes ENABLE ROW LEVEL SECURITY;
ALTER TABLE feature_permissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_feature_overrides ENABLE ROW LEVEL SECURITY;

```

```

-- (all new tables)

-- Feature permissions: readable by all committee (executive+); writable by admin
only
CREATE POLICY "feature_perms_read" ON feature_permissions
  FOR SELECT USING (
    EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid()
           AND portal_role IN ('executive','secretary','president'))
  );

CREATE POLICY "feature_perms_write" ON feature_permissions
  FOR ALL USING (
    EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid()
           AND is_admin = true)
  );

```

RLS test requirement: before go-live, test with role-specific JWTs:

- **anon** JWT: must be blocked from all sensitive tables
- **member** JWT: must be blocked from **maintenance_records**, **votes** (others)
- **executive** JWT: must be blocked from phone numbers in **profiles**; must not be able to write to **feature_permissions**
- **secretary** JWT: must be able to read **feature_permissions**; must not be able to write to it (admin-only write)
- **is_admin = false** with **secretary** JWT: must not access admin-only endpoints

16.3 DPDP Act Compliance

- Privacy policy at utamacs.org/privacy before any data collection
- Explicit consent checkbox on first login; consent stored in **privacy_consent**s
- Purpose limitation: member data used only for governance communications
- Data breach: affected members notified within 72 hours

16.4 Feature Permission API Security

The **feature_permissions** and **user_feature_overrides** tables drive UI rendering, but **every API route independently re-checks permissions** from the database:

```

// In every API route
const { feature, user } = await resolvePermission(request,
  'hoto.approve_president');
if (!feature.enabled) {
  return Response.json({ error: 'FORBIDDEN' }, { status: 403 });
}

```

This means: even if a client somehow renders a button that should be hidden, the API rejects the request. UI gating is UX; API checking is security.

17. Data Migration Sprint

17.1 Pre-HOTO Migration (May 6-25)

Priority	Source	Content
P1	Committee email inboxes	Builder letters, NOCs, certificates from Ankura Homes
P1	Google Drive	Ascenza scope documents, inspection reports
P1	WhatsApp	Photos from site inspections
P2	Physical	Scans of handover letters, signed agreements
P3	WhatsApp	Meeting minutes, decisions (screenshot → PDF)

Tools built for migration:

- Admin bulk import: `/portal/admin/import` — CSV creates multiple items in one operation
- Bulk document upload: multiple files → `upload_queue` → associated with HOTO item
- Every migrated document gets `source_description` = "Migrated from [source] by [person] on [date]"

17.2 HOTO Item Seeding (May 6-9)

80+ items seeded from Ascenza scope CSV before committee training:

```
# seeds/seed-hoto-items.ts
# Reads: seeds/hoto-items.csv
# Inserts hoto_items + hoto_required_docs
# Commits _index.json to governance-data
```

18. Role-Based Access Control & Feature Permissions

18.1 Feature Registry

Every controllable feature in the system has a canonical name. The feature registry is the source of truth:

```
// src/lib/features.ts

export const FEATURES = {
  // User Management
  'users.view_directory': { label: 'View member directory', locked: false },
  'users.invite_member': { label: 'Invite new members', locked: false },
  'users.invite_committee': { label: 'Invite committee members', locked: true }, // president only
  'users.change_role': { label: 'Change member roles', locked: true }, // president only
  'users.deactivate': { label: 'Deactivate members', locked: false },

  // HOTO
  'hoto.view': { label: 'View HOTO items', locked: true }, // all committee
  'hoto.create': { label: 'Create/edit HOTO items', locked: false },
  'hoto.upload': { label: 'Upload documents', locked: false }
}
```

```

false },
  'hoto.comment':          { label: 'Add comments',                locked:
false },
  'hoto.advance_status':   { label: 'Advance item status',          locked:
false },
  'hoto.approve_president': { label: 'President approval gate',      locked:
true }, // president; or executive with VP title + active delegation
  'hoto.approve_secretary': { label: 'Secretary approval gate',      locked:
true }, // secretary; or executive with Joint Secretary title + active delegation
  'hoto.bypass_required_docs': { label: 'Bypass required document gate', locked:
true }, // secretary and president

  // Snag
  'snag.view':             { label: 'View snag list',                locked:
true },
  'snag.create':           { label: 'Create/edit snags',              locked:
false },
  'snag.delete':           { label: 'Delete snags',                  locked:
true }, // president only
  'snag.verify_close':      { label: 'Mark snags as verified closed', locked:
true }, // secretary and president

  // Vendor
  'vendor.view':           { label: 'View vendor evaluations',        locked:
true },
  'vendor.view_quotes':    { label: 'View vendor quotes',            locked:
false },
  'vendor.vote':           { label: 'Cast vendor vote',              locked:
false },
  'vendor.open_voting':     { label: 'Open/close voting',             locked:
false },
  'vendor.final_select':    { label: 'Confirm final vendor selection', locked:
true }, // secretary and president

  // Finance
  'finance.view':          { label: 'View financial records',        locked: false },
  'finance.enter':          { label: 'Enter maintenance/expense records',
locked: false },
  'finance.approve_10k':    { label: 'Approve expenses ≤₹10K ($9.11a)',
locked: true }, // secretary and president
  'finance.approve_20k':    { label: 'Approve expenses ≤₹20K ($9.11a)',
locked: true }, // president only
  'finance.open_board_vote': { label: 'Open Board resolution vote ≤₹50K ($9.11b)',
locked: true }, // secretary only – starts the vote
  'finance.view_member_phones': { label: 'View member phone numbers',
locked: true }, // secretary and president

  // Notices
  'notice.view':           { label: 'View formal notices',            locked:
false },
  'notice.send':           { label: 'Send formal notices',            locked:
false },

  // Admin
  'admin.delegation':       { label: 'Manage delegation settings',      locked:
true }, // president only
  'admin.elections':        { label: 'Run committee election update',    locked:

```

```

true  }, // president only
  'admin.permissions':      { label: 'Manage feature permissions',      locked:
true  }, // president only
  'admin.import':           { label: 'Bulk data import',                locked:
false },

  // Audit
  'audit.view':             { label: 'View audit log',                  locked:
false },
} as const;

export type Feature = keyof typeof FEATURES;

```

locked: true features have their default role assignment enforced by the system and cannot be changed by the admin permissions UI. **locked: false** features can be toggled per role.

18.2 Default Role Permissions

Four roles only: **member**, **executive**, **secretary**, **president**. The **committee_title** field (e.g., "Treasurer", "Vice President") is display-only and has **no effect** on this table. Finance access for Treasurer title-holders and delegation-based approvals for VP / Joint Secretary are handled via **user_feature_overrides** (§18.5), not by role.

```

export const DEFAULT_ROLE_PERMISSIONS: Record<string, Feature[]> = {
  member: [
    'hoto.view',
    'snag.view',
    'vendor.view',
    'notice.view',
  ],

  executive: [
    // HOTO – create and participate, but cannot approve
    'hoto.view', 'hoto.create', 'hoto.upload', 'hoto.comment',
    'hoto.advance_status',
    // Snag – create items, cannot verify-close or delete
    'snag.view', 'snag.create',
    // Vendor – view quotes and cast votes, cannot open voting or select finalist
    'vendor.view', 'vendor.view_quotes', 'vendor.vote',
    // Notice – read only
    'notice.view',
    // Audit – read-only trail
    'audit.view',
    // NOTE: finance.view / finance.enter are NOT included here.
    // They are granted as user_feature_overrides when committee_title =
    'Treasurer'
    // or 'Joint Treasurer' is assigned (prompted by election/role-change
    workflow).
  ],

  secretary: [
    // All executive features +
    'hoto.view', 'hoto.create', 'hoto.upload', 'hoto.comment',
    'hoto.advance_status',
    'hoto.approve_secretary', 'hoto.bypass_required_docs',
    'snag.view', 'snag.create', 'snag.verify_close',

```

```

    'vendor.view', 'vendor.view_quotes', 'vendor.vote',
    'vendor.open_voting', 'vendor.final_select',
    'notice.view', 'notice.send',
    'audit.view',
    'finance.view', 'finance.enter', 'finance.approve_10k',
    // finance.open_board_vote: opens a board resolution vote for ₹20,001–₹50,000.
    // Uses vendor_requirements (category='FINANCIAL_APPROVAL') + votes table.
    // On majority+quorum, API auto-creates corpus_fund_records APPROVED_USE entry
    ($9.3).
    'finance.open_board_vote',
    'finance.view_member_phones',
    'users.view_directory', 'users.invite_member', 'users.deactivate',
  ],

  president: [
    // All secretary features +
    'hoto.view', 'hoto.create', 'hoto.upload', 'hoto.comment',
    'hoto.advance_status',
    'hoto.approve_secretary', 'hoto.approve_president',
    'hoto.bypass_required_docs',
    'snag.view', 'snag.create', 'snag.verify_close', 'snag.delete',
    'vendor.view', 'vendor.view_quotes', 'vendor.vote',
    'vendor.open_voting', 'vendor.final_select',
    'notice.view', 'notice.send',
    'audit.view',
    'finance.view', 'finance.enter', 'finance.approve_10k', 'finance.approve_20k',
    'finance.open_board_vote', 'finance.view_member_phones',
    'users.view_directory', 'users.invite_member', 'users.deactivate',
    'users.invite_committee', 'users.change_role',
    'admin.delegation', 'admin.elections', 'admin.permissions', 'admin.import',
  ],
};

// Delegation note: when an executive holds committee_title = 'Vice President' and
// is the
// active target in approval_delegations, the system grants
// 'hoto.approve_president' as a
// transient user_feature_override for the delegation period. Likewise for Joint
// Secretary
// → 'hoto.approve_secretary'. Neither changes portal_role. See §8.2 / §8.4.

```

18.3 Permission Resolution at Runtime

Every page load and every API call resolves permissions through this function:

```

// src/lib/permissions.ts

export async function resolveUserPermissions(
  userId: string,
  societyId: string
): Promise<Set<Feature>> {
  // 1. Get user's current role
  const { data: profile } = await supabase
    .from('profiles')
    .select('portal_role')
    .eq('id', userId)

```



```

    .single();

    // 2. Get role-level permissions from feature_permissions table
    //     (falls back to DEFAULT_ROLE_PERMISSIONS if no DB override)
    const { data: rolePerms } = await supabase
      .from('feature_permissions')
      .select('feature, enabled')
      .eq('society_id', societyId)
      .eq('role', profile.portal_role);

    // 3. Get user-specific overrides (non-revoked, non-expired)
    const { data: userOverrides } = await supabase
      .from('user_feature_overrides')
      .select('feature, enabled')
      .eq('society_id', societyId)
      .eq('user_id', userId)
      .is('revoked_at', null)
      .or(`expires_at.is.null,expires_at.gt.${new Date().toISOString()}`);

    // 4. Build final permission set: role defaults → role overrides → user
    overrides
    const permissions = new Set<Feature>
    (DEFAULT_ROLE_PERMISSIONS[profile.portal_role]);

    for (const perm of rolePerms ?? []) {
      perm.enabled ? permissions.add(perm.feature as Feature)
        : permissions.delete(perm.feature as Feature);
    }
    for (const override of userOverrides ?? []) {
      override.enabled ? permissions.add(override.feature as Feature)
        : permissions.delete(override.feature as Feature);
    }

    return permissions;
  }

  export function can(permissions: Set<Feature>, feature: Feature): boolean {
    return permissions.has(feature);
  }

```

18.4 UI Enforcement Pattern

In every Astro page component:

```

---
// src/pages/portal/hoto/[id].astro
const user = await requireAuth(Astro);
const permissions = await resolveUserPermissions(user.id, user.society_id);
---

<!-- Buttons only render if the user has the feature -->
{can(permissions, 'hoto.upload') && (
  <button class="btn-primary">Upload Document</button>
)}

{can(permissions, 'hoto.approve_president') && item.status === 'PENDING_PRESIDENT'

```

```
&& (
  <button class="btn-primary btn-large">APPROVE</button>
)}

{can(permissions, 'hoto.advance_status') && (
  <StatusTransitionButtons item={item} />
)}

<!-- Disabled with tooltip for features the user almost-qualifies for -->
{!can(permissions, 'hoto.approve_president') && item.status ===
'PENDING_PRESIDENT' && (
  <div class="permission-notice">
    Waiting for President's approval
  </div>
)}
```

Rules for UI enforcement:

- Hidden entirely: features the user's role would never have
- Disabled with explanation: features temporarily unavailable (e.g., "Voting closes in 3 days" vs "You've already voted")
- Visible but blocked: approval gates where the user sees the item but cannot act (shows who needs to act)
- Never show a 403 error page for in-page features — always render the read-only state with a clear label

18.5 Admin Feature Permissions UI (/portal/admin/permissions)

Only the **admin** (`is_admin = true`) can access this page. The admin uses it to apply a feature permission change that the President has authorized. Every save requires the authorization form to be completed.

FEATURE PERMISSIONS

[Viewing: Executive Role ▼]

HOTO Module

View HOTO items

Always ON (locked – all committee)

Create/edit HOTO items

ON [Toggle]

Upload documents

ON [Toggle]

Add comments

ON [Toggle]

Advance item status

ON [Toggle]

President approval gate

Always ON for president only (locked)

Secretary approval gate

Always ON for secretary only (locked)

Snag Module

View snag list

Always ON (locked – all committee)

Create/edit snags

ON [Toggle]

Delete snags

Always ON for president only (locked)

Verify-close snags

Always ON for president/secretary (locked)

Vendor Module

View vendor evaluations

Always ON (locked – all committee)

View vendor quotes

ON [Toggle]

Cast vendor vote

ON [Toggle]

Open/close voting

ON [Toggle]

Finance Module

View financial records

OFF [Toggle]

Enter records

OFF [Toggle]

50 / 82

 Approve ≤₹10K	Always ON for secretary only (locked)
 Approve ≤₹20K	Always ON for president only (locked)

[Save Changes for Executive Role] [Reset to Defaults]

Locked features display a tooltip: "This permission is mandated by the society's byelaws and cannot be changed."

Non-locked features can be toggled per role by the admin with documented President authorization. Examples of legitimate changes:

- Give **executive** role access to **finance.view** (see totals, not enter data)
- Give an executive with Joint Treasurer title access to **notice.send** for a specific operational period (via per-user override, §18.6)
- Remove **vendor.view_quotes** from **executive** during a sensitive procurement

18.6 Per-User Feature Overrides

For edge cases where one person needs temporary access outside their role. The admin grants these with documented President authorization.

/portal/admin/users/[id] → Permissions tab

[Name] (Flat 204) – Role: Executive

INHERITED FROM EXECUTIVE ROLE:

-  hoto.view, hoto.create, hoto.upload, hoto.comment ...
-  finance.view (not in executive defaults)

USER-SPECIFIC OVERRIDES:

finance.view	ENABLED	Expires: 2026-08-31
Reason: Acting financial coordinator during Treasurer's absence		
Granted by: Admin on 2026-06-15		[Revoke]

[+ Add Override] ← Admin only

Feature: [finance.view ▼] Enable/Disable: [Enable ▼]

Reason: _____ Expires: ____/____/____ (optional)

[Grant Override]

Overrides:

- Require a mandatory reason (for audit trail)
- Optional expiry date (auto-revoked when expired)
- Shown in the user's audit trail
- Can be revoked at any time by the admin
- Only the admin can grant or revoke overrides

Guiding principle: Overrides are for genuine edge cases (temporary acting role, specific investigation). They are not a substitute for proper role management via the election workflow.

18.7 Feature Access Summary Matrix

Two orthogonal axes: **governance role** (`portal_role` — approval/voting/action power) and **admin flag** (`is_admin=true` — system management power). The `committee_title` field is display-only and does **not** change any column in this matrix.

Special cases handled via `user_feature_overrides` (not shown as separate columns):

- **Treasurer / Joint Treasurer title** → grants `finance.view` + `finance.enter` to an `executive`
- **VP / Joint Secretary title + active delegation** → grants `hoto.approve_president` / `hoto.approve_secretary` to an `executive` for the delegation period

Feature	member	executive	secretary	president	admin (<code>is_admin=true</code>)
User Management — admin only					
View member directory	-	-	✓	✓	✓
Invite new member	-	-	✓	-	✓
Invite committee member	-	-	-	-	✓
Change any governance role	-	-	-	-	✓
Set <code>committee_title</code>	-	-	-	-	✓
Deactivate / reactivate member	-	-	-	-	✓
Run election bulk update	-	-	-	-	✓
Manage feature permissions per role	-	-	-	-	✓
Grant / revoke per-user feature override	-	-	-	-	✓
Grant / revoke admin flag	-	-	-	-	✓
Manage delegation settings	-	-	-	-	✓
HOTO — governance role controls these					
View items	R	R	R	R	R
Create / edit items	-	✓	✓	✓	via governance role
Upload documents	-	✓	✓	✓	via governance role
Advance status	-	✓	✓	✓	via governance role
President approval gate	-	✓ ¹	-	✓	—
Secretary approval gate	-	✓ ²	✓	✓	—
Bypass required docs gate	-	-	✓	✓	—
Snag					
View	R	R	R	R	R
Create / edit	-	✓	✓	✓	via governance role

Feature	member	executive	secretary	president	admin (<i>is_admin=true</i>)
Verify-close	-	-	✓	✓	—
Delete (soft)	-	-	-	✓	—
Vendor					
View evaluations	R	R	R	R	R
View quotes	-	✓	✓	✓	via governance role
Cast vote	-	✓	✓	✓	via governance role
Open voting	-	-	✓	✓	—
Final selection confirm	-	-	✓	✓	—
Finance					
View records	-	✓ ³	✓	✓	—
Enter records	-	✓ ³	✓	✓	—
Approve ≤₹10K	-	-	✓	✓	—
Approve ≤₹20K	-	-	-	✓	—
Open board vote (₹20K–₹50K)	-	-	✓	-	—
View member phone numbers	-	-	✓	✓	✓
Notices					
View	R	R	R	R	R
Send / publish	-	-	✓	✓	—
Audit					
View audit log	-	✓	✓	✓	✓
View role-change history (all users)	-	-	✓	✓	✓

Footnotes:

- ¹ *executive* gets *hoto.approve_president* only when *committee_title* = 'Vice President' or 'Working President' **and** an active *approval_delegations* row points to them. Granted as a time-bounded *user_feature_override*.
- ² *executive* gets *hoto.approve_secretary* only when *committee_title* = 'Joint Secretary' and an active *approval_delegations* row points to them.
- ³ *executive* gets *finance.view* and *finance.enter* only when *committee_title* = 'Treasurer' or 'Joint Treasurer'. Granted as *user_feature_overrides* during the role-assignment workflow (§5.5).

Reading the matrix:

- president* has no user-management ✓ marks — the President governs; the admin administers. These are orthogonal.
- Admin accountability flows from the audit log (every action permanently recorded) and from Secretary/President viewing role-change history — not from in-portal approval workflows.

- An admin who also holds a governance role (e.g., **secretary**) gets both sets of capabilities simultaneously.

19. Document Management

19.1 Required Document Prompting

Each HOTO item has a **required_documents** list. Cannot advance to **UNDER_REVIEW** if required docs missing — hard gate. Secretary or President can bypass with mandatory written reason (stored in **hoto_required_docs.bypass_reason** and audit log).

19.2 Document Metadata (Every Upload)

- **uploaded_by**: server-resolved from session (not client-provided)
- **uploaded_at**: server timestamp (not client clock)
- **file_hash_sha256**: computed server-side before uploading
- **source_description**: "Received from Ankura Homes on 2026-04-15"
- **github_sha**: the git commit SHA — cryptographically verifiable

19.3 Document Versioning

New version uploaded → old version gets **superseded_by = new_id**. Old version stays downloadable. Version history shown on document card.

19.4 Confidentiality Rules

Data	Visible to
Member phone numbers	secretary and president (plus executives with <code>finance.view_member_phones</code> override)
Vendor quotes	All committee (executive and above)
Legal notices	All committee
Financial records	secretary and president; executives with Treasurer/Joint Treasurer title (<code>finance.view</code> override)
Proxy documents	secretary, president
Audit log	executive and above (all committee)

20. Scope Boundary

20.1 What This Platform Covers (v1)

1. User & role management (invite, role assignment, elections, feature permissions)
2. HOTO tracking — Ascenza-scope items from June 1, 2026
3. Snag list management — common area; individual apartments as courtesy log
4. Vendor evaluation and selection — current 5 active evaluations
5. Maintenance collection tracking — from May 1, 2025
6. Corpus fund and expense tracking
7. Formal notice generation to builder and members
8. MC governance: voting, approvals, delegation

20.2 Explicitly Out of Scope (v1)

Committed to `governance-data/_meta/scope-v1.md` before launch. When scope-creep requests arrive, this is the documented response:

- Clubhouse/amenity booking
- Resident-to-resident contact directory
- Maintenance complaint routing for residents
- Individual apartment snagging resolution
- Annual General Body Meeting workflow
- Gate access / visitor management
- WhatsApp Business API integration

21. Phase-wise Implementation Plan

21.0 Non-Regression Principles

These rules govern the entire implementation. Every sprint task is subject to them. They are not optional and are not overridden by timeline pressure.

Rule 1: No existing feature is touched unless the design explicitly requires a functional change to it.

The new HOTO, Snag, Vendor, Finance, Notice, and RBAC modules are **additions** to the existing portal. They live alongside what already exists. No existing page, component, API route, or database table is deleted, renamed, or restructured as part of this work unless it is explicitly called out in the design with a stated reason.

What "explicitly required" means:

- The design document names the specific file or table and explains why the change is needed
- The change is a bug fix for an existing defect (e.g., `user_roles` unique constraint)
- The change is a necessary extension to an existing system to support new functionality (e.g., adding columns to `profiles`)
- The change is an additive navigation addition (new items added to the portal menu, not replacing existing ones)

What does NOT qualify:

- "While I'm in this file, I'll clean up X" — stop; leave X as-is
- "This component could be refactored to be cleaner" — not in scope
- "This old pattern conflicts with the new approach" — maintain both; document the conflict; resolve it only if it causes an actual breakage

Rule 2: Database migrations are additive only.

Allowed	Not Allowed Without Explicit Review
<code>CREATE TABLE IF NOT EXISTS</code>	<code>DROP TABLE</code>
<code>ALTER TABLE ADD COLUMN IF NOT EXISTS</code>	<code>DROP COLUMN</code>
<code>CREATE INDEX IF NOT EXISTS</code>	<code>ALTER COLUMN TYPE</code>
<code>ADD CONSTRAINT IF NOT EXISTS</code> (with pre-check)	<code>TRUNCATE</code>
<code>CREATE OR REPLACE FUNCTION</code>	Renaming any existing table or column

For the `user_roles` unique constraint specifically: Before applying the constraint, run this query to confirm there are no duplicates in production:

```
SELECT user_id, society_id, COUNT(*)
FROM user_roles
GROUP BY user_id, society_id
HAVING COUNT(*) > 1;
```

If duplicates exist, resolve them manually first, then apply the constraint. A failed constraint migration can lock the table and break login for all users.

Rule 3: Existing features are tested before and after every sprint that touches shared code.

Critical paths to test before closing any sprint:

Feature	Test	File(s)
Letter generation (new)	Create a multi-page letter; verify first page layout, subsequent pages have no offset	<code>src/pages/portal/letters/new.astro</code>
Letter generation (edit)	Open existing letter, edit, download PDF	<code>src/pages/portal/letters/[id].astro</code>
Portal login	Log in with an existing committee member account	Supabase auth
Admin password reset	Trigger and complete a password reset	<code>/portal/admin/</code>
Public website navigation	Nav and footer load on all public pages	<code>src/components/nav.html</code> , <code>footer.html</code>
Existing portal layout	Portal layout renders correctly with any new nav items added	<code>src/layouts/PortalLayout.astro</code>

These tests are done manually before a sprint is considered complete. If any existing feature is broken by a change, the sprint is not done until it is fixed.

Rule 4: New navigation items are additions, not replacements.

When adding new portal modules to the navigation (HOTO, Snags, Vendors, Finance, Notices, Admin), the new items are appended to the existing navigation structure. No existing nav item is removed, reordered, or renamed unless the design explicitly requires it.

Known existing navigation items to preserve:

- Dashboard (existing)
- Letters (`/portal/letters/`)
- Admin (`/portal/admin/`)
- Any other items currently in `PortalLayout.astro`

Known fix pending (not a new addition — pre-existing bug):

- The **compliance** module nav item currently maps to `/portal/admin/audit` instead of `/portal/admin`; this should be corrected but is not this project's concern unless it is explicitly in the sprint scope.

Rule 5: New columns on **profiles** use defaults that don't break existing rows.

Every new column added to the **profiles** table must have a default value that is valid for existing rows:

Column	Default	Why it's safe
<code>is_admin</code>	<code>false</code>	Existing users do not become admins unexpectedly
<code>portal_role</code>	<code>'executive'</code>	Existing committee member accounts keep their current effective access
<code>payment_status</code>	<code>'current'</code>	Existing rows not flagged as defaulters
<code>is_active</code>	<code>true</code>	Existing accounts remain active
<code>privacy_consent_given</code>	<code>false</code>	Existing users prompted to consent on next login

The privacy consent prompt for existing users: On first login after the consent feature goes live, existing users see the consent screen. They cannot access the portal until they accept. This is intentional (DPDP Act requirement) but the impact on existing users must be communicated to them before deployment — not just discovered when they try to log in.

Rule 6: The feature permission system defaults to "everything already working continues to work."

When the **feature_permissions** table is seeded, every feature that existing roles could already access must be set to `enabled = true` for those roles. No existing capability is silently revoked by seeding default permissions.

Verification query after seeding:

```
-- Check: no existing committee member has lost access to a feature they had before
-- Expected: all committee-level features enabled for executive and above
SELECT role, feature, enabled
FROM feature_permissions
WHERE enabled = false
ORDER BY role, feature;
-- Review this list carefully before go-live: every disabled entry is a deliberate choice
```

Rule 7: The letter generation system is extended, not replaced.

The formal notice auto-generation (Module 5) reuses the existing PDF generation infrastructure. The approach:

- The existing `/portal/letters/new.astro` and `/portal/letters/[id].astro` pages remain fully functional and unchanged for manual letter creation
- The notice auto-draft feature calls the existing letter generation logic programmatically, passing the item data as the template context
- The result is stored in GitHub at `notices/drafts/` — distinct from manually created letters stored elsewhere

- If the existing letter generation logic needs to be extracted into a shared utility for reuse, that refactor is done as a standalone commit with the existing letter feature tested before and after

Rule 8: The `is_admin` flag does not interfere with existing auth checks.

Existing auth checks in the portal use `portal_role` (or the existing `user_roles` table). The new `is_admin` flag is an additional check only — it is never a replacement for existing role checks. Pattern:

```
// Existing check (unchanged):
if (user.portal_role !== 'president' && user.portal_role !== 'secretary') {
  return redirect('/portal');
}

// New admin check (additive — in new pages only):
if (!user.is_admin) {
  return redirect('/portal');
}
```

Existing pages that check `portal_role` are not modified to also check `is_admin`. New admin-only pages check `is_admin`. The two systems are independent.

Urgency Context

Today is May 6, 2026. HOTO starts June 1, 2026. That is **26 days**. The plan below compresses the critical path to a 3-week emergency sprint.

Principle: Imperfect digital tracking from Day 1 is worth more than a perfect system 6 weeks later.

Emergency Sprint — May 6 to May 31 (25 days)

Goal: System live May 31. All 14 members onboarded with correct roles. 80+ HOTO items seeded. Bal Reddy has done one walkthrough.

Scope IN:

- Infrastructure: governance-data repo, GitHub App, upload queue, health-check cron, Supabase migrations
- User management: invite-only registration, role assignment, user directory, role change with audit trail
- Feature permissions: default role permissions seeded; admin permissions UI
- Auth: existing Supabase auth + new role columns + RLS on all new tables
- HOTO item list and detail: view, comment, upload (queued), required doc alerts, status transitions
- Mobile "My Actions" screen with Approve / Not Yet for President
- Email notifications: action required, status change (Resend)
- Seeded: all 80+ HOTO items + required doc checklists
- Security: pre-commit hook, RLS test, anon key blocked
- Privacy: consent checkbox on first login, privacy policy live

Scope OUT (deferred to Phase 2):

- Full President+Secretary approval workflow gates (simplified: Secretary manually marks "submitted for approval")
- Snag module bulk CSV import

- Vendor voting
- Financial tracking
- Formal notice generation
- Election bulk update workflow
- Per-user feature overrides

Days	Deliverable	Notes
May 6-9 (Days 1-4)	Create <code>utamacs/governance-data</code> repo. Set up GitHub App; install key in Vercel env vars. Add pre-commit hook. Run Supabase migration 027 (all new tables including <code>member_invites</code> , <code>role_change_log</code> , <code>election_events</code> , <code>feature_permissions</code> , <code>user_feature_overrides</code>). Fix <code>user_roles</code> unique constraint bug.	Critical infrastructure
May 9 (Day 4)	Health-check cron + Supabase keepalive cron running. Verify both write to <code>github_api_log</code> .	Non-negotiable
May 10-12 (Days 5-7)	User directory: <code>/portal/admin/users</code> — list with role badges, payment status, last active. Pending invites tab. Role change UI (dropdown + mandatory reason + confirmation + email notification + audit log).	RBAC management core
May 12-14 (Days 8-9)	Invite flow: admin sends invite → Resend email with one-time link → registration with privacy consent → account created → admin notified.	Needed before onboarding 14 members
May 14-17 (Days 10-12)	Seed default <code>feature_permissions</code> for all roles. Admin permissions UI at <code>/portal/admin/permissions</code> (view defaults; toggle non-locked features per role).	Needed before access is given
May 17-21 (Days 13-16)	HOTO item list: filterable by category/status/priority. HOTO item detail: required doc checklist, document upload via <code>upload_queue</code> with status polling, comment thread, status timeline, transitions with role checks.	Core HOTO tracker
May 21-24 (Days 17-19)	"My Actions" mobile screen. Simplified approve/reject UI for non-tech users. Email notifications (action required, status changed) via Resend.	President adoption
May 24-27 (Days 20-22)	Invite and onboard all 14 committee members with correct roles. Verify each role's UI shows correct features (spot-check with executive and secretary accounts).	Committee onboarding
May 27-29 (Days 23-24)	Data migration P1: builder letters, NOCs, Ascenza scope docs uploaded. Privacy policy live at <code>utamacs.org/privacy</code> . <code>scope-v1.md</code> and <code>voting-policy.md</code> committed to <code>governance-data</code> .	Legal/compliance
May 29-30 (Day 25)	Pre-launch walkthrough with Bal Reddy: views My Actions, reads an item, approves. Fix any confusion. Final smoke test on mobile.	President sign-off

Days	Deliverable	Notes
May 31	Go-live. All 14 members active. HOTO tracking begins.	

Phase 2 — June 1 to June 28 (4 weeks)

Week	Deliverable
Week 5 (Jun 1-7)	Full President + Secretary approval workflow with delegation. Delegation management UI. Acting-on-behalf tagging.
Week 6 (Jun 8-14)	Snag list CRUD. Snag scope with liability disclaimer. Bulk CSV import from Ascenza. Photo upload from mobile camera.
Week 7 (Jun 15-21)	Committee election bulk update workflow at /portal/admin/elections . Per-user feature overrides UI.
Week 8 (Jun 22-28)	Vendor evaluation module: board, vendor profiles, comparison matrix, voting with quorum enforcement, conflict of interest, proxy, decision record.

Phase 3 — July 1 to July 31 (4 weeks)

Week	Deliverable
Week 9 (Jul 1-7)	Maintenance collection tracking. Defaulter tracking with thresholds. Monthly defaulter list generation and email.
Week 10 (Jul 8-14)	Corpus fund tracker with overdraft prevention. Expense tracker with byelaw authority enforcement. Builder dues register.
Week 11 (Jul 15-21)	Formal notice generation. Auto-draft at 30-day SLA breach. RERA escalation tracker.
Week 12 (Jul 22-31)	Post-selection vendor tracking. Async PDF generation (job queue).

Phase 4 — August 1 to August 31 (4 weeks)

Week	Deliverable
Week 13 (Aug 1-14)	Mobile UX audit. Storage health tile. Resident fortnightly HOTO digest email.
Week 14 (Aug 15-22)	Security review: RLS audit, feature permission enforcement test, corpus fund overdraft test.
Week 15 (Aug 23-28)	RUNBOOK.md validated: Secretary follows it without assistance.
Week 16 (Aug 29-31)	Retrospective. Phase 5 scope assessment (AGM, meeting management, WhatsApp notifications).

Pre-Launch Checklist (Must Be Done Before May 31)

Infrastructure

- ☐ `utamacs/governance-data` private repo created
- ☐ GitHub App installed; private key in Vercel env vars only (not in any file)
- ☐ Pre-commit hook blocking private key commits installed and tested
- ☐ Supabase migration 027 run and verified in production
- ☐ `user_roles` unique constraint added (fix for existing bug)
- ☐ Health-check cron running every 15 minutes
- ☐ Supabase keepalive cron running every 6 days
- ☐ Upload queue cron running every 60 seconds
- ☐ Vercel Pro upgrade completed (14-minute timeout)
- ☐ Resend DNS for `utamacs.org` verified

Security & Privacy

- ☐ RLS enabled on all new tables; anon key tested with zero sensitive access
- ☐ `executive` JWT tested: cannot see `maintenance_records` or member phone numbers
- ☐ `member` JWT tested: cannot access financial or governance documents
- ☐ Privacy policy live at `utamacs.org/privacy`
- ☐ Consent checkbox on first login; test it blocks registration until accepted

User Management

- ☐ Default `feature_permissions` seeded for all roles (from `DEFAULT_ROLE_PERMISSIONS`)
- ☐ Admin permissions UI verified: locked features cannot be toggled
- ☐ All 14 committee members invited and accepted (with correct roles)
- ☐ Invite flow tested end-to-end: invite → email → register → login → correct features visible

Content & Compliance

- ☐ 80+ HOTO items seeded with required document checklists
- ☐ P1 historical documents uploaded with `source_description`
- ☐ `governance-data/_meta/scope-v1.md` committed
- ☐ `governance-data/_meta/voting-policy.md` committed

UX Validation

- ☐ Bal Reddy walkthrough completed: viewed item, approved, saw vote screen
- ☐ Working President tested on mobile browser
- ☐ All role-specific screens spot-checked (executive, secretary, president)

22. Comprehensive Risk Register

22.1 RBAC & User Management Risks (New in v3.1)

Risk	Probability	Impact	Mitigation
Role change via UI silently fails (existing bug)	High	High	<code>user_roles</code> unique constraint fix in migration 027; tested before go-live
Admin accidentally gives wrong role at election	Medium	High	Election bulk update preview screen shows all changes before confirm; atomic transaction
Feature permission toggle disables a byelaw-required gate	Medium	Critical	Locked features cannot be toggled; tooltip explains why; API independently checks regardless

Risk	Probability	Impact	Mitigation
Invite link forwarded to unauthorized person	Low	High	One-time token; invalidated immediately after use; email-verified only
Auto-reassignment misses an item	Low	Medium	Query covers ALL assigned items; tested before go-live; orphan dashboard tile shows unassigned items
Per-user override granted inappropriately	Low	Medium	Only President can grant overrides; mandatory reason stored; shown in user's audit trail
UI shows feature but API blocks it (desync)	Low	High	API independently resolves permissions from DB on every call; UI and API use same <code>resolveUserPermissions</code> function
Feature permissions DB record missing (new role)	Medium	Medium	Default role permissions are hardcoded in <code>DEFAULT_ROLE_PERMISSIONS</code> ; DB overrides are additive

22.2 Technical Risks

Risk	Probability	Impact	Mitigation
GitHub API rate limit during bulk session	Medium	High	<code>upload_queue</code> cron, max 30/batch
Vercel 10s timeout kills PDF	Medium	High	Async <code>pdf_generation_jobs</code> ; Vercel Pro backup
GitHub App token silent failure	Medium	Critical	15-min health-check; Resend alert on failure
Supabase DB pauses	Medium	Medium	6-day keepalive cron
RLS gaps expose PII	Medium	High	RLS test suite with role JWTs before go-live
Private key leaked to git	Low	Critical	Pre-commit hook; Vercel env vars only

22.3 Adoption Risks

Risk	Probability	Impact	Mitigation
President doesn't adopt portal	High	Critical	Non-tech UX spec; "My Actions" default; pre-launch walkthrough mandatory
Committee reverts to WhatsApp	High	Medium	Financial payments require portal approval record
Committee member dropout mid-HOTO	Medium	High	Role-based assignment; orphan auto-escalation

22.4 Governance & Legal Risks

Risk	Probability	Impact	Mitigation
Vote challenged (proxy invalidity)	Low	High	Notarized proxy doc linked to vote; policy pre-committed

Risk	Probability	Impact	Mitigation
RERA document metadata challenged	Low	High	Server timestamps; SHA-256 hash; source description
DPDP Act compliance gap	Low	High	Privacy policy + consent + consent timestamp
Byelaw ambiguity mid-implementation	Medium	Medium	governance_notes field; interpretation documented not encoded

22.5 Operational Risks

Risk	Probability	Impact	Mitigation
Phase 1 too late for June 1	High	Critical	Emergency sprint; minimum viable scope explicitly defined
Data migration never completed	High	Medium	Dedicated migration sprint May 6-25; P1 priority enforced
Architecture bus-factor	High	High	RUNBOOK.md; Secretary validates before go-live
Scope creep from residents	High	Medium	scope-v1.md committed to governance-data

22.6 Financial Risks

Risk	Probability	Impact	Mitigation
Corpus fund overdraft	Low	Critical	Server-side balance check; API rejects if insufficient
Expense exceeds byelaw authority	Low	High	API enforces \$9.11 limits; blocks with byelaw citation
Builder dues never collected	Medium	High	SLA escalation cron; RERA notice auto-draft at 30 days

22.7 The Three That Kill the Project

If only three risks get managed before everything else:

1. **Phase 1 timing** — Cut scope; May 31 non-negotiable. A late system means HOTO starts with no digital trail. That trail cannot be reconstructed retroactively.
2. **President adoption** — The "My Actions" mobile screen must be so simple Bal Reddy prefers it. Tested in the pre-launch walkthrough, not assumed.
3. **RBAC correctness** — If the role change UI silently fails (existing bug), or if features show for roles that shouldn't have them, the audit trail is meaningless and approvals cannot be trusted. Fix the **user_roles** constraint on Day 1; test every role's feature visibility before go-live.

23. Rules Engine

23.1 Design Philosophy

v3.x hardcoded all business logic — approval limits, escalation timelines, notification recipients, validation constraints — directly in API routes. When requirements change (byelaw amendments, operational adjustments, new committee practices), every change requires a code deployment.

The v4.0 Rules Engine moves all configurable logic into the database, fully managed through the admin UI. The engine covers five rule categories:

Category	What it controls	Who can change
PARAMETER	Numeric thresholds, durations, feature flags	Admin (locked rules need byelaw amendment)
APPROVAL	Who approves what, chain ordering, delegation	Admin only
ESCALATION	Item type × days overdue → action	Admin only
NOTIFICATION	Event → recipients → channel	Admin only
VALIDATION	Field constraints, state transition guards	Admin only

Important distinction:

- **Structural rules** (the *existence* of dual sign-off, one-apartment-one-vote, quorum requirement) are byelaw text. They stay in code. Changing them requires a formal byelaw amendment, a society resolution, and a code deployment.
- **Parametric rules** (the *values* — ₹10,000, 90 days, 8 directors) are data. They live in the **rules** table and are managed through the UI.

The engine does not replace code judgment — it provides values that code uses to make decisions.

23.2 Rules Table

```
CREATE TABLE rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  society_id UUID REFERENCES societies NOT NULL,
  rule_category TEXT NOT NULL, -- 'PARAMETER', 'APPROVAL', 'ESCALATION',
  'NOTIFICATION', 'VALIDATION'
  rule_code TEXT NOT NULL,
  label TEXT NOT NULL,
  description TEXT,
  byelaw_reference TEXT,
  value_type TEXT NOT NULL, -- 'integer', 'decimal', 'boolean', 'integer_array',
  'date_string', 'json'
  current_value JSONB NOT NULL,
  default_value JSONB NOT NULL,
  is_locked BOOLEAN DEFAULT true, -- locked = needs formal byelaw amendment to
  change
  effective_from DATE NOT NULL DEFAULT CURRENT_DATE,
  changed_by UUID REFERENCES profiles,
  changed_at TIMESTAMPTZ DEFAULT NOW(),
  change_reason TEXT,
  UNIQUE (society_id, rule_code)
);
CREATE INDEX idx_rules_society ON rules(society_id, rule_category, rule_code);
```


23.3 Complete Rules Registry (Seed Values)

Category: PARAMETER — numeric and boolean configuration values

rule_code	Label	Byelaw	Default	Locked
QUORUM_REQUIRED	Board quorum (directors required to vote)	§7.16(a)	8	✓
TOTAL_DIRECTORS	Total number of directors	§7.16(a)	14	✓
VOTE_SUSPENSION_DAYS	Maintenance arrears days before vote suspended	§4.6	90	✓
DEFAULTER_FLAG_DAYS	Days before "Defaulting Member" flag	§6.36	60	✓
DEFAULTER_NOTICE_DAYS	Days before services denial warning	§6.37	90	✓
MAINTENANCE_INTEREST_RATE	Annual interest on arrears (% p.a.)	§19(e)	18	✓
SECRETARY_APPROVAL_LIMIT	Max expense Secretary can approve unilaterally (₹)	§9.11(a)	10000	✓
PRESIDENT_APPROVAL_LIMIT	Max expense President can approve unilaterally (₹)	§9.11(a)	20000	✓
BOARD_APPROVAL_LIMIT	Max expense requiring Board vote (₹)	§9.11(b)	50000	✓
MINUTES_SUBMISSION_DAYS	Days to submit meeting minutes	§7.16(e)	7	✓
ANNUAL_STATEMENT_DEADLINE	Annual financial statement deadline (MM-DD)	§9.3	"09–30"	✓
INVITE_EXPIRY_DAYS	Invite link validity (days)	—	7	×
PROXY_VOTING_ENABLED	Allow proxy authorization for vendor votes	—	false	×
UPLOAD_MAX_SIZE_MB	Maximum file upload size (MB)	—	5	×
PDF_PURGE_DAYS	Days after which PDF job input_data is PII-scrubbed	—	30	×
PROXY_EXPIRY_ALERT_DAYS	Days before proxy expiry to alert admin	—	2	×
EMAIL_DRAFT_RETENTION_DAYS	Days to retain SENT/DISCARDED email drafts	—	365	×

Category: APPROVAL — who approves what and in what order

rule_code	Label	Default value	Locked
HOTO_APPROVAL_CHAIN	Roles required to approve HOTO items (in order)	["secretary","president"]	✓
HOTO_APPROVAL_ALTERNATE_VP	Executive with VP title substitutes for President when	true	✓

rule_code	Label	Default value	Locked
	delegation is active		
HOTO_APPROVAL_ALTERNATE_JOINT_SEC	Executive with Joint Secretary title substitutes for Secretary when delegation is active	true	✓
VENDOR_DECISION_REQUIRES_BOTH	Vendor final selection requires President + Secretary	true	✓
EXPENSE_APPROVAL_CHAIN_10K	Role(s) who can approve expenses ≤ limit	["secretary"]	✓
EXPENSE_APPROVAL_CHAIN_20K	Role(s) who can approve expenses ≤ limit	["president"]	✓
EXPENSE_APPROVAL_CHAIN_50K	Board vote required for expenses ≤ limit	"BOARD_VOTE"	✓

Category: ESCALATION — overdue triggers and actions

rule_code	Label	Default value	Locked
HOTO_SLA_ESCALATION_DAYS	Days overdue before escalation actions	[7,14,30]	×
HOTO_SLA_DAY7_ACTION	Action at 7 days overdue	"EMAIL_COMMITTEE"	×
HOTO_SLA_DAY14_ACTION	Action at 14 days overdue	"EMAIL_URGENT_FLAG"	×
HOTO_SLA_DAY30_ACTION	Action at 30 days overdue	"AUTO_DRAFT_NOTICE"	×
SNAG_SLA_WARNING_DAYS	Days before snag builder-committed date to warn	7	×
DEFAULTER_REMINDER_DAYS	Days arrears before reminder email to member	30	×
PENDING_APPROVAL_REMINDER_HOURS	Hours before re-notifying approver of pending item	48	×
PROXY_EXPIRY_ALERT_DAYS	Days before proxy expiry to alert admin	2	×

Category: NOTIFICATION — who gets notified of what

rule_code	Label	Default value	Locked
NOTIFY_HOTO_APPROVAL_NEEDED	Recipients when HOTO needs approval	["approver"]	×
NOTIFY_VOTE_OPENED	Recipients when vendor vote opens	["all_committee"]	×
NOTIFY_BUILDER_SLA_OVERDUE	Recipients when builder SLA overdue	["committee"]	×
NOTIFY_GITHUB_HEALTH_FAIL	Recipients for storage outage alert	["admin","secretary"]	×
NOTIFY_ELECTION_COMPLETE	Recipients after election bulk update	["all_affected","president","secretary"]	×
WEEKLY_DIGEST_ENABLED	Send weekly HOTO digest to committee	true	×
WEEKLY_DIGEST_DAY	Day of week for weekly digest (0=Sun)	1	×
WEEKLY_DIGEST_HOUR	Hour (24h) to send weekly digest	7	×

Category: **VALIDATION** — state transition guards and field constraints

rule_code	Label	Default value	Locked
HOTO_REQUIRE_DOCS_BEFORE_REVIEW	Block UNDER_REVIEW if required docs missing	true	×
VOTE_REQUIRE_CONFLICT_DECLARATION	Force conflict-of-interest declaration before vote	true	✓
PAYMENT_REQUIRE_ELECTRONIC_ABOVE	Min amount (₹) requiring electronic payment mode	10000	✓
HOTO_EVIDENCE_REQUIRED_BEFORE_UPLOAD	Must select an HOTO item before uploading doc	true	×
SNAG_SCOPE_REQUIRED_ON_CREATE	snag_scope mandatory on snag creation	true	×
INVITE_EMAIL_DOMAIN_ALLOWLIST	Restrict invites to specific email domains (empty = any)	[]	×

23.4 Runtime Access Pattern

```
// src/lib/rules.ts

// Single-rule fetch with typed fallback
export async function getRule<T>({
  societyId: string,
  ruleCode: string,
  fallback: T
}): Promise<T> {
  const { data } = await supabase
    .from('rules')
    .select('current_value')
    .eq('society_id', societyId)
    .eq('rule_code', ruleCode)
    .single();
  return (data?.current_value as T) ?? fallback;
}

// Batch-load all rules for a society (cached per request)
export async function loadRules(societyId: string): Promise<Map<string, unknown>> {
  const { data } = await supabase
    .from('rules')
    .select('rule_code, current_value')
    .eq('society_id', societyId);
  return new Map((data ?? []).map(r => [r.rule_code, r.current_value]));
}

// Helper: get rule from pre-loaded map (zero DB calls after initial load)
export function r<T>(rules: Map<string, unknown>, code: string, fallback: T): T {
  return (rules.get(code) as T) ?? fallback;
}

// Usage – batch load once per API route, use throughout:
const rules = await loadRules(societyId);
```

```
// Financial limit check:
const limit = r(rules, 'SECRETARY_APPROVAL_LIMIT', 10000);
if (amount > limit) {
  return Response.json({
    error: 'APPROVAL_LIMIT_EXCEEDED',
    message: `Your approval authority is up to ₹${limit.toLocaleString('en-IN')}
    (Byelaw §9.11a). This expense needs the President's approval.`,
    limit, amount
  }, { status: 422 });
}

// Quorum check:
const quorum = r(rules, 'QUORUM_REQUIRED', 8);
if (voteCount < quorum) {
  return { quorumMet: false, required: quorum, current: voteCount };
}

// Approval chain resolution:
const chain = r(rules, 'HOTO_APPROVAL_CHAIN', ['secretary','president']);
const nextApprover = chain[currentApprovalStep];

// Escalation check:
const escalationDays = r(rules, 'HOTO_SLA_ESCALATION_DAYS', [7, 14, 30]);
const actions = escalationDays.map((days, i) => ({ days, action: r(rules,
'HOTO_SLA_DAY${days}_ACTION', '') }));

// Notification recipients:
const recipients = r(rules, 'NOTIFY_BUILDER_SLA_OVERDUE', ['committee']);
// Expand 'committee' → fetch all committee members from profiles
```

Caching strategy: `loadRules()` is called once per API route invocation and the result is passed as a parameter — no request-scoped cache needed. The batch SELECT fetches all society rules in one query. Rules change infrequently; stale reads by seconds are acceptable (noted in UI tooltip: "Changes take effect on the next action").

23.5 Rules Engine Admin UI — </portal/admin/rules>

Admin-only management. Non-admin committee can view (read-only) to understand what rules are in force.

The UI has five tabs — one per rule category.

RULES ENGINE	Admin access	[View Change History]
Tabs: [Parameters] [Approval] [Escalation] [Notification] [Validation]		

Tab 1 — Parameters

PARAMETERS		
BYELAW-MANDATED (🔒 Locked – require formal amendment to change)		
🔒 Board quorum required	8 of 14 directors	§7.16(a)

 Vote suspension threshold	90 days arrears	\$4.6
 Secretary approval limit	₹10,000	\$9.11(a)
 President approval limit	₹20,000	\$9.11(a)
 Board approval limit	₹50,000	\$9.11(b)
 Maintenance interest rate	18% per annum	\$19(e)
 Minutes deadline	7 days after meeting	\$7.16(e)

OPERATIONAL (Admin-configurable)

Invite link validity	7 days	[Edit]
Proxy voting	 Disabled	[Enable]
Max upload file size	5 MB	[Edit]
Builder SLA warnings	7, 14, 30 days	[Edit]
PDF data scrub after	30 days	[Edit]
Proxy expiry alert	2 days before	[Edit]
Email draft retention	1 year	[Edit]

Tab 2 — Approval Chains

APPROVAL CHAINS

HOTO ITEMS

Who approves (in order): Secretary → President [Edit Order]
 If Secretary absent: Joint Secretary steps in [Toggle] 
 If President absent: Vice President (when delegated) [Toggle] 

VENDOR DECISIONS

Final selection requires: President AND Secretary (both) [Locked 

EXPENSES

≤ Secretary limit (₹10,000): Secretary alone [Locked 
 ≤ President limit (₹20,000): President alone [Locked 
 ≤ Board limit (₹50,000): Board vote (quorum 8/14) [Locked 
 > Board limit: API blocks – not possible [Locked 

Editing an approval chain shows a drag-to-reorder interface for the chain order (unlocked chains only). A preview shows: "HOTO items will require approval from: Secretary (Step 1) → President (Step 2)."

Tab 3 — Escalation Rules

ESCALATION RULES

HOTO ITEMS – Builder SLA Overdue

7 days overdue → Email all committee (urgent flag) [Edit]
 14 days overdue → Email with URGENT banner [Edit]
 30 days overdue → Auto-draft formal notice + RERA flag [Edit]

Each trigger: action [EMAIL_COMMITTEE ▼] [+ Add Trigger] [x Remove]

SNAG ITEMS – Builder Committed Date

7 days before → Remind Secretary [Edit]
 Past date → Email Secretary + flag in dashboard [Edit]

MAINTENANCE DEFAULTERS		
30 days arrears →	Email reminder to member	[Edit]
60 days arrears →	Flag committee dashboard	[Edit]
90 days arrears →	Suspend vote rights; 7-day notice	[Locked 🔒]
PENDING APPROVALS		
Re-notify approver after:	48 hours of no action	[Edit]

Each escalation trigger is editable (threshold days + action type). Action types available: **EMAIL_COMMITTEE**, **EMAIL_URGENT_FLAG**, **AUTO_DRAFT_NOTICE**, **DASHBOARD_FLAG**, **EMAIL_MEMBER**, **SUSPEND_VOTE_RIGHTS**. Locked triggers (byelaw-mandated) cannot be removed or have their action changed.

Tab 4 — Notification Recipients

NOTIFICATION RECIPIENTS			
Event	Recipients	Channel	
HOTO needs President approval	President only	Email	[Edit]
HOTO needs Secretary approval	Secretary only	Email	[Edit]
Vendor vote opened	All committee	Email	[Edit]
Builder SLA overdue	Committee	Email	[Edit]
GitHub storage down	Admin + Secretary	Email	[Edit]
Election completed	All affected members	Email	[Edit]
Weekly HOTO digest	All committee	Email	
Enabled: ☒	Day: Monday	Hour: 7:00 AM	[Edit]
Recipient options: approver / all_committee / committee / secretary / president / admin / all_affected / uploader / member			

Each row is editable. Adding recipients beyond the default widens notification scope; reducing them narrows it. Locked rows (e.g., "GitHub down → admin") cannot be narrowed below the minimum required for the system to function.

Tab 5 — Validation Rules

VALIDATION RULES		
HOTO		
☒	Block status advance to UNDER_REVIEW if required docs missing	[Toggle]
☒	Evidence document required before status advance	[Toggle]
SNAGS		
☒	snag_scope (common/individual) required on creation	[Toggle]
VENDORS		
☐	Conflict-of-interest declaration required before voting	[Locked]
☐	voting_policy_committed must be true before votes open	[Locked]
FINANCE		
☐	Electronic payment required for amounts > ₹10,000	[Locked]
☐	Cash payments blocked	[Locked]

USERS

Invite email domain allowlist (empty = any domain): [] [Save]

Toggle-type validations can be turned on/off. Locked validations are byelaw requirements.

Interaction Rules (All Tabs)

1. **Editing:** Click [Edit] → inline field with current value. Enter new value + mandatory reason text.
2. **Confirmation:** "You are changing [rule] from [old] to [new]. Reason: '[text]'. This takes effect on the next action — in-flight operations use the previous value."
3. **On save:** rules.current_value updated; changed_by, changed_at, change_reason recorded; audit_log entry created.
4. **Change History:** Global log showing: who changed what, old value → new value, reason, timestamp. Cannot be deleted.
5. **Reset to Default:** Each unlocked rule has a [Reset] button. Requires a reason ("Reverting to byelaw default after temporary change").
6. **Locked rule hover:** "This rule is set by the society's registered byelaws (TG/RRD/MACS/2026-15/FOW & M). It can only be changed if the byelaws are formally amended at a General Body Meeting and a new registration is obtained."

24. Email Management & Draft System

24.1 Three-Tier Email Model

Not all system emails should send automatically. Formal communications to external parties or members require human review before sending. The three tiers:

Tier	Examples	Behaviour
1 — Operational	File upload complete, PDF ready, session expired, invite accepted	Auto-send immediately via Resend
2 — Action Required	Approval needed, vote opened, delegation changed, health alert	Auto-send with action button; tracked in dashboard
3 — Formal Draft	Builder notices, defaulter notices, RERA packages, formal letters	Created as DRAFT; Secretary or President must review and send

Tier 1 and 2 send immediately. Tier 3 never sends automatically — it creates a draft that waits for human review.

24.2 Email Drafts Table

```
CREATE TABLE email_drafts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  society_id UUID REFERENCES societies NOT NULL,  
  tier INTEGER NOT NULL CHECK (tier IN (1, 2, 3)),  
  triggered_by TEXT NOT NULL,           -- event type: 'SLA_30_DAY', 'DEFAULTER_90D',  
'VOTE_CLOSED'  
  trigger_resource_type TEXT,           -- 'hoto_item', 'snag_item',  
'vendor_requirement', 'member'  
  trigger_resource_id TEXT,  
  recipient_type TEXT NOT NULL,         -- 'BUILDER', 'MEMBER', 'COMMITTEE', 'ADMIN'  
  recipient_email TEXT,
```



```
recipient_name TEXT,
subject TEXT NOT NULL,
body_html TEXT NOT NULL,
body_text TEXT NOT NULL,
suggested_sender_name TEXT NOT NULL,
suggested_sender_email TEXT NOT NULL,
status TEXT DEFAULT 'DRAFT',          -- 'DRAFT', 'REVIEWED', 'SENT', 'DISCARDED'
reviewed_by UUID REFERENCES profiles,
reviewed_at TIMESTAMPTZ,
sent_by UUID REFERENCES profiles,
sent_at TIMESTAMPTZ,
resend_message_id TEXT,
discarded_by UUID REFERENCES profiles,
discarded_reason TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_email_drafts_status ON email_drafts(society_id, status,
created_at DESC);
```

24.3 Trigger → Draft Mapping

Trigger	Tier	Recipient	Subject Template
Builder SLA 7 days overdue	2	Committee	"OVERDUE: [item] builder deadline was [date]"
Builder SLA 14 days overdue	2	Committee	"URGENT: [item] now 14 days overdue"
Builder SLA 30 days overdue	3	Builder	"Legal Notice — [item] — Urban Trilla MACS"
HOTO item needs President approval	2	President	"ACTION REQUIRED: [item] needs your approval"
HOTO item needs Secretary approval	2	Secretary	"ACTION REQUIRED: [item] needs your approval"
Vendor vote opened	2	All committee	"VOTE OPEN: [vendor category] — closes [date]"
Vendor vote result	2	All committee	"VOTE RESULT: [vendor selected/failed quorum]"
Defaulter 90+ days	3	Member	"Maintenance Default Notice — Urban Trilla MACS"
GitHub health 3 failures	1	Admin + Secretary	"URGENT: Governance storage unavailable"
Upload PERMANENTLY_FAILED	1	Uploader + Admin	"File upload failed — action needed"
Invite accepted	1	Admin	"[Name] (Flat [N]) has accepted their invitation"
Weekly HOTO digest	2	All committee	"HOTO Week [N] Summary — [X] items pending"

24.4 Draft Review UI — </portal/admin/email-drafts>

Secretary and President can access. Nav badge shows pending count:  2.

PENDING EMAIL DRAFTS (2)



READY TO SEND

Generated today, 3:45 PM

Legal Notice – HOTO-042 Lift AMC (30 days overdue)

To: Ankura Homes <legal@ankurahomes.com>

Subject: Legal Notice – HOTO Item Overdue – Urban Trilla MACS

[Preview Full Email]

[Edit Subject / Body]

[Send Now]

[x Discard]



READY TO SEND

Generated yesterday

Maintenance Default Notice – Flat 312 (92 days overdue)

To: [Member Name] <email@example.com>

Subject: Maintenance Default Notice – UTA MACS (Byelaw \$6.37)

[Preview Full Email]

[Edit Subject / Body]

[Send Now]

[x Discard]

Preview: Renders the HTML email exactly as it appears in an inbox — sender name, subject, body, footer. The reviewer sees what the recipient will see.

Edit: Inline editor for subject and body only. The system fills in recipient email and sender credentials automatically from the society profile — reviewers cannot redirect emails or spoof senders.

Send: Calls Resend API with the stored `body_html`; records `sent_by`, `sent_at`, `resend_message_id`. Status → **SENT**. Cannot be unsent — the audit trail records the send.

Discard: Requires a reason. Status → **DISCARDED**. Item stays in history for audit purposes.

24.5 Weekly Digest Email

Generated every Monday at 7:00 AM by cron `generate-weekly-digest`. Content:

Subject: UTA MACS – HOTO Week [N] Summary (May 26–June 1, 2026)

Greetings Committee Members,

HOTO PROGRESS THIS WEEK

Items completed: 4

Items advanced: 12

Currently overdue: 3 (builder-dependent)

PENDING ACTIONS

- Lift AMC Transfer – Waiting for President approval (3 days)
- KONE Service Contract – Documents missing
- Water Tank OC – Builder notified; awaiting response

- VENDOR DECISIONS
- Property Management: Voting open until June 5
 - Accounting Tool: Selection pending Board vote

- UPCOMING DEADLINES
- Terrace Waterproofing SLA: 8 days remaining
 - Backup DG Commissioning: 14 days remaining

[Open Portal →]

Urban Trilla MACS | portal.utamacs.org

Sent as Tier 2 (auto-send to all committee). The draft is created, sent immediately, and recorded in `email_drafts` with `status = 'SENT'` for audit purposes.

25. RBAC Administration UI — Complete Specification

25.1 Three-Area Structure

The RBAC admin UI has three distinct areas, each with its own URL and purpose:

Area	URL	Purpose	Who accesses
Role-Feature Matrix	<code>/portal/admin/permissions</code>	Which features each role has	Admin only
User Role Assignment	<code>/portal/admin/users</code>	Which role each user holds	Admin only
Per-User Overrides	<code>/portal/admin/users/[id]/permissions</code>	Exceptions for individual users	Admin only

25.2 Role-Feature Matrix (`/portal/admin/permissions`)

The primary RBAC configuration interface. Shows a full matrix of roles × features, with visual toggle cells.

FEATURE PERMISSIONS MATRIX

View mode: [Matrix ▼]

[Preview as User...]

[Export CSV]

member

exec

sec

pres

LEGEND

USER MANAGEMENT (admin controls these via the admin flag – not shown here)

HOTO MODULE

View items

Create/edit

Upload docs

default)

Add comments

Advance status

user_feature_override

Pres. approval gate

Sec. approval gate

🔒

✓

–

–

–

–

–

🔒

–

🔒

🔒

✓

✓

✓

✓

✓

✓

–†

–†

🔒

✓

✓

✓

✓

✓

–

–

–

🔒

✓

✓

✓

✓

✓

–

–

✓

🔒

✓

✓

✓

✓

✓

–

–

✓

🔒

✓

✓

✓

✓

✓

–

–

✓

🔒 = locked (byelaw)

✓ = enabled

– = disabled (role default)

† = via

only; not this matrix

75 / 82

Bypass required-docs gate			—		—		✓		✓	
SNAG MODULE										
View		✓		✓		✓		✓		✓
Create/edit	—		✓		✓		✓		✓	
Verify-close		—		—		✓		✓		✓
Delete (soft)		—		—		—		—		✓
VENDOR MODULE										
View evaluations		✓		✓		✓		✓		✓
View quotes	—		✓		✓		✓		✓	
Cast vote	—		✓		✓		✓		✓	
Open voting	—		—		✓		✓		✓	
Confirm final selection		—		—		✓		✓		✓
FINANCE MODULE										
View records	—	—†			✓		✓		✓	
Enter records	—	—†			✓		✓		✓	
Approve ≤₹10K		—		—		✓		✓		✓
Approve ≤₹20K		—		—		—		—		✓
Open Board vote (≤₹50K)		—		—		✓		✓		—
View member phone numbers		—		—		✓		✓		✓
NOTICES										
View		✓		✓		✓		✓		✓
Send / publish	—		—		✓		✓		✓	
AUDIT										
View audit log	—		✓		✓		✓		✓	
View role-change history	—		—		✓		✓		✓	
[Save All Changes]			[Reset to Defaults]			[View Change Log]				

† Delegation overrides (for Pres./Sec. approval gates) and Treasurer finance access are granted as user_feature_overrides (§25.4), not via this matrix. Editing them here has no effect — a tooltip explains this when the cell is hovered.

Interaction rules:

- Click an unlocked cell (✓/—) to toggle it; it turns amber to indicate an unsaved change
- Clicking a locked () cell shows a tooltip: "This is required by the society's byelaws and cannot be changed here."
- Clicking a delegation-override cell (†) shows: "This access is granted per-user via delegation or title assignment. Manage it in User → Per-User Overrides."
- "Save All Changes" shows a diff summary modal before committing: "You are enabling 'Finance → View records' for Executive and disabling 'Vendor → View quotes' for Executive. These changes take effect immediately."
- "Preview as User" → select any user → screen shows exactly what buttons and sections that user would see on the HOTO, Snag, Vendor, and Finance pages

25.3 Role Assignment with Visual Hierarchy (/portal/admin/users)

Beyond the list view in §5.9, the admin has a **Roles View** tab showing who holds each governance role and their display title. The **role** determines authority; the **committee_title** is the label shown in the directory.

COMMITTEE ROLES		[+ Invite Member]	[Run Election]
	PRESIDENT		
	Bal Reddy (Flat 101) · President	[Edit]	[Change Role]
	SECRETARY		
	[Name] (Flat 312) · General Secretary	[Edit]	[Change Role]
	EXECUTIVES (9 members)		
	[Name] (Flat 204) · Vice President	[Edit]	[Change Role]
	[Name] (Flat 108) · Joint Secretary	[Edit]	[Change Role]
	[Name] (Flat 207) · Treasurer 	[Edit]	[Change Role]
	[Name] (Flat 415) · Joint Treasurer 	[Edit]	[Change Role]
	[Name] (Flat 306) · Working President	[Edit]	[Change Role]
	[Name] (Flat 512) · Executive Member	[Edit]	[Change Role]
	[Name] (Flat 203) · Executive Member	[Edit]	[Change Role]
	... 2 more		
	MEMBERS (48) – shown in list view only		
	ADMIN FLAG (orthogonal to governance roles)		
	System Admin	[Your name] (Flat –)	[Manage Admins]
 = has <code>finance.view</code> + <code>finance.enter</code> overrides (Treasurer title grants these)			

[Edit] title flow (title-only change, no permission impact):

1. Click [Edit] → inline text input appears for `committee_title`; role dropdown grayed out (use [Change Role] for that)
2. Admin types new title (e.g., "Vice President"), presses Save
3. `role_change_log` created with `change_type = 'TITLE_ONLY'`; no permission change; user notified

[Change Role] flow (role change, may affect permissions):

1. Click [Change Role] → drawer slides in showing:
 - Current role badge + current title
 - Role dropdown (4 choices: member / executive / secretary / president)
 - Committee title field (pre-filled from current title; admin can update)
 - Reason field (mandatory)
 - Permission diff preview: lists features being added and removed
2. If new title = 'Treasurer' or 'Joint Treasurer': system prompts "Grant `finance.view` + `finance.enter` overrides for this user? [Yes — grant] [No — skip]"
3. If new role = 'executive' from higher role: "This removes `hoto.approve_secretary` and/or financial approval access. Confirm?"
4. On confirm: role updated, title updated, overrides granted if elected, `role_change_log` created, email sent to user

25.4 Per-User Permission Overrides (`/portal/admin/users/[id]/permissions`)

Two-panel layout showing inherited + overrides:

[Name] (Flat 207) – Executive • Treasurer

INHERITED FROM EXECUTIVE ROLE

✓	hoto.view	
✓	hoto.create	
✓	hoto.upload	
✓	hoto.comment	
✓	hoto.advance_status	
✓	snag.view	
✓	snag.create	
✓	vendor.view	
✓	vendor.view_quotes	
✓	vendor.vote	
✓	audit.view	
✗	finance.view	← role default: ✗ override → ✓ ENABLED
✗	finance.enter	← role default: ✗ override → ✓ ENABLED
✗	finance.approve_10k	(not in role)
✗	finance.approve_20k	(not in role)

USER-SPECIFIC OVERRIDES

finance.view ENABLED
Reason: Treasurer title –
finance access per §5.5
Granted: Admin, Jun 15
Expires: never
[x Revoke]

finance.enter ENABLED
Reason: Treasurer title
Granted: Admin, Jun 15
Expires: never
[x Revoke]

[+ Add Override]

Effective: finance.view = ✓ ENABLED (override wins over role default)
Override expiry is enforced server-side on every API call – not just login.

[+ Add Override] form:

- Feature: dropdown of all FEATURES
- Enable or Disable: radio (enable grants access above role; disable restricts below role)
- Reason: required text (for audit trail)
- Expires: optional date picker (blank = no expiry)
- On save: `user_feature_overrides` record created; user receives email notification

25.5 Admin Flag Management (`/portal/admin/users/admins`)

Separate sub-page for managing who holds `is_admin = true`:

SYSTEM ADMINISTRATORS

[Name] Flat – Admin since: Jun 1, 2026 [Revoke Admin]

No one else has admin access.

[+ Grant Admin Access]

Enter email of existing member → confirm → `is_admin = true`

⚠ Only grant admin access to people you fully trust.
Admins can change any role, enable any feature, and deactivate any member. This cannot be undone without another admin.

- Only another admin can grant or revoke the admin flag

- Cannot revoke your own admin flag (prevents lock-out)
- Must have at least one admin at all times (API enforces)

26. Post-Redesign Regression Analysis (v4.0 / v4.1 Self-Check)

This section verifies that the six new design areas introduced in v4.0 do not create new gaps, contradict existing sections, or break existing functionality.

26.1 Byelaw Rules Engine — Regression Checks

Check	Status	Note
Existing hardcoded values match seed defaults	✓	₹10K/₹20K/₹50K/quorum=8/90-day all match
<code>getRuleValue</code> is a read-only DB call — no side effects	✓	SELECT only; safe in API middleware
Locked rules cannot be changed via the admin UI	✓	<code>is_locked = true</code> → edit disabled; API enforces
Rules engine UI is admin-only	✓	<code>is_admin = true</code> gate; non-admin sees read-only view
Rule change is logged in <code>audit_log</code>	✓	Change recorded with <code>changed_by</code> , <code>change_reason</code>
Fallback values in <code>getRuleValue</code> match byelaw defaults	✓	Hardcoded fallbacks = seed defaults = byelaw text
Non-regression: §2 tables unchanged	✓	§2 references rules engine parameters by name

26.2 Async Resilience — Regression Checks

Check	Status	Note
Existing <code>upload_queue</code> behaviour preserved	✓	Backoff fields are additive; <code>null backoff_until</code> = no backoff (old behaviour)
Circuit breaker does not affect read operations	✓	Only blocks <code>process-uploads</code> cron; views/downloads unaffected
<code>cron_locks</code> UNIQUE constraint cannot deadlock normal operation	✓	10-min expiry auto-releases stale locks
DLQ dashboard does not expose PII	✓	Shows file path and error only; no member data
Retry does not re-process already-committed uploads	✓	Only <code>status = 'PENDING'</code> items are picked up; COMPLETED items excluded
<code>system_config</code> table is a new addition — no existing FK dependencies	✓	New table; no existing migrations affected

26.3 Email Draft System — Regression Checks

Check	Status	Note
Tier 1 and 2 emails continue to auto-send	✓	Only Tier 3 creates a DRAFT; existing auto-send behaviour unchanged
Existing Resend integration unchanged	✓	Email drafts use same Resend client; new <code>sent_by</code> + <code>sent_at</code> tracking added
No email sent to builder without human review	✓	All builder communications are Tier 3 DRAFT
Weekly digest does not send if no committee members are active	✓	Cron checks <code>profiles.is_active = true</code> before generating recipient list
<code>email_drafts</code> table does not store raw passwords or tokens	✓	Stores only rendered subject+body + recipient email
Secretary/President see email drafts; admin also sees	✓	Role check in draft review UI

26.4 Proxy Disabled by Default — Regression Checks

Check	Status	Note
<code>proxy_authorizations</code> table not dropped or altered	✓	Table retained; feature is gated, not removed
When <code>PROXY_VOTING_ENABLED = false</code> , proxy upload UI hidden	✓	UI conditional on <code>getRuleValue('PROXY_VOTING_ENABLED')</code>
When <code>false</code> , API rejects proxy vote submissions with clear message	✓	Server-side check before processing vote
Admin can enable at any time — takes effect immediately	✓	Rules engine update → next page load shows proxy option
Existing proxy votes (if any) are not retroactively invalidated when disabled	✓	Disable affects new votes only; existing records unchanged

26.5 RBAC UI Expansion — Regression Checks

Check	Status	Note
Matrix UI is display-only until "Save All Changes" clicked	✓	No partial saves; atomic update of all changed cells
Preview-as-User uses same <code>resolveUserPermissions</code> function as production	✓	Same code path — no separate preview logic that could diverge
Role assignment change emails to affected user	✓	Existing §5.5 flow preserved
Vacant role warnings are advisory, not blocking	✓	Admin can proceed without filling VP; warning only
Admin cannot revoke own admin flag	✓	API enforces minimum-1-admin rule
Non-regression: §18 RBAC text and §25 new spec are consistent	✓	§25 is the UI spec; §18 is the logic spec; no contradictions

26.6 New Gaps Identified During Regression

These gaps surfaced during the v4.0 self-check and are documented for resolution in implementation:

1. **system_config table needs definition** — circuit breaker state is stored there but the table schema is not defined. Add to §15 schema.
2. **Email draft retention** — `email_drafts` rows are never deleted. Define a retention policy (e.g., purge `SENT/DISCARDED` rows older than 1 year via a cron).
3. **getRuleValue caching strategy** — "30s TTL per request context" needs a concrete implementation path (Vercel edge cache or a module-level Map with timestamp).
4. **Weekly digest opt-out** — if a committee member does not want weekly emails, there is no unsubscribe mechanism. Add `email_digest_enabled BOOLEAN DEFAULT true` to `profiles`.
5. **Rules engine change propagation** — if `SECRETARY_APPROVAL_LIMIT` is changed while an expense approval is in-flight, the in-flight approval uses the old value (cached at approval-start time). This is acceptable behaviour but should be documented in the UI tooltip: "Changes take effect on the next submission."

These 5 items are low-priority (no blockers) and can be addressed in Phase 2.

Document Version 4.1 · Revised May 2026 — 4-Role Model + committee_title Display Field

Changes from v4.0: Reduced portal_role from 9 values to 4 (member, executive, secretary, president); added committee_title TEXT to profiles as a display-only label with no permission effect; VP/Working President/Joint Secretary collapse to executive + title + delegation-based feature override; Treasurer/Joint Treasurer collapse to executive + title + finance.view/finance.enter user_feature_overrides; §3.4 committee structure rewritten; §5.4 role hierarchy simplified; §5.5 role change workflow updated (role dropdown now 4 choices + title field); §5.6 election bulk update updated (Role + Title as separate columns; finance override prompt); §5.9 user directory updated (role badge + title below); §6.2 HOTO state machine role references updated; §15 schema: committee_title column + full data migration SQL (2-step collapse); §16.2 RLS updated for 4-role names; §18.1 locked feature comments updated; §18.2 DEFAULT_ROLE_PERMISSIONS rewritten to 4-role structure with delegation note; §18.7 Feature Access Matrix rewritten from 7 columns to 4 columns with footnote system; §23.3 APPROVAL rules labels updated for delegation via title; §25.2 RBAC matrix updated to 4 columns with + delegation override markers; §25.3 role assignment UI shows role + title as separate editable fields; §25.4 per-user override example updated to Executive · Treasurer; RUNBOOK §5 and §6 updated for 4-role + title workflow; RUNBOOK §7 delegation steps note that VP/Joint Secretary are executives with titles

Changes from v3.2: §23 general-purpose Rules Engine (5 categories: PARAMETER, APPROVAL, ESCALATION, NOTIFICATION, VALIDATION) with 35+ configurable rules and full tab-based admin UI; §24 Email Draft System (3-tier model, email_drafts table, review UI, weekly digest spec, 13-event trigger mapping); §25 full RBAC Administration UI spec (role-feature matrix with visual toggles, role assignment with hierarchy view, per-user overrides with two-panel layout, admin flag management); §26 post-redesign regression analysis (6 categories, 5 new gaps identified and documented); §4.5 Async Resilience Patterns (exponential backoff, circuit breaker, idempotency locks, cron heartbeat monitoring, DLQ dashboard); §4.4 updated to reference new RUNBOOK.md; proxy voting disabled by default via rules engine; new DB tables: rules, email_drafts, cron_heartbeats, cron_locks, system_config; upload_queue backoff fields; profiles.email_digest_enabled; design/RUNBOOK.md created (20 sections, 600+ lines)

Changes from v3.1: Added societies table (FK anchor for all society_id columns); clarified member_invites.accepted_user_id FK creation timing and token timing-attack prevention; added responsible_role/responsible_user_id to snag_items; added reopen_reason to snag_items; specified proxy authorization expiry enforcement mechanism with proactive cron alert; defined Corpus Fund APPROVED_USE approval chain (presidential vs board vs blocked); changed health-check cron from write-based (96 commits/day) to read-only GET; added path traversal validation spec for upload_queue.target_github_path; added finance.open_board_vote feature for Board resolution votes ≤₹50K; specified board financial vote mechanism reusing vendor_requirements+votes tables with

FINANCIAL_APPROVAL category; added *pdf_generation_jobs.purged_at* with 30-day PII scrub retention policy; added §12.5 Validation Messages & Blocked Feature UX (full message catalog, display patterns, non-tech user vote screen, My Actions overflow/pagination, no-403-pages rule); added corpus fund §9.3 with three-tier approval chain table

Changes from v3: Added Module 0 (User & Role Management); invite-only registration; committee election bulk update; member deactivation; auto-reassignment; feature registry with locked/unlocked features; runtime permission resolution; UI enforcement pattern; admin permissions UI; per-user feature overrides; fixed user_roles unique constraint bug; RBAC risk register; pre-launch checklist

Based on: Registered Byelaws TG/RRD/MACS/2026-15/FOW & M · Ascenza HOTO Scope · Committee Q&A · Risk Analysis · RBAC Requirements · Final Design Review (21 findings)

Next review: Post-Phase 1 go-live (June 2026)